

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA**



**BÁO CÁO BÀI TẬP LỚN
MÔ HÌNH HÓA TOÁN HỌC**

Invigilator Assignment Problem (IAP)

Giảng viên hướng dẫn: Nguyễn An Khương,
Lê Hồng Trang,
Phạm An Vinh

Nhóm: 9

Lớp: L01

Tp. Hồ Chí Minh, Tháng 5/2026

BẢNG PHÂN CÔNG CÁC THÀNH VIÊN

Bảng 0.1 BẢNG PHÂN CÔNG VÀ ĐÁNH GIÁ TIẾN ĐỘ CÔNG VIỆC

STT	Thành viên	Chi tiết công việc	% HT
1	Nguyễn Duy Khánh	- Phần 1: Phân tích tham số. - Phần 2: Biến mục tiêu. - Phần 3: Hàm mục tiêu. - Viết báo cáo 1, 2, 3, kết luận. - Kiểm tra tổng thể.	100%
2	Phan Quốc Huy	- Phần 4: Hàm ràng buộc cứng. - Phần 5: Hàm ràng buộc mềm. - Phần 10: Thuật toán đề xuất. - Làm slide, hỗ trợ làm phần sau latex.	100%
3	Nguyễn Ngọc Anh Khoa	- Phần 6: Đọc dataset, xử lý data. - Phần 7: Xây dựng mô hình ILP tại các tool được đề xuất, tích hợp solver vào code. - Phần 8: Chạy solver, tuning nếu infeasible. - Viết tuning: xử lý khi solver báo lỗi. - Viết README.md, requirements.txt.	100%
4	Trần Quỳnh Anh	- Phần 9: Đánh giá độ phức tạp, thời gian thực thi, so sánh với baseline. - Viết phần Appendix, Meeting minutes.	100%

MỤC LỤC

BẢNG PHÂN CÔNG CÁC THÀNH VIÊN	1
1 Giới thiệu về bài toán phân công giám thị (IAP)	6
1.1 Tổng quan bài toán	6
1.2 Dữ liệu thực tế	6
1.3 Mục tiêu của đề tài	7
1.4 Phạm vi và giả thuyết	7
2 Mô hình hóa toán học	8
2.1 Phân tích tập hợp và tham số hệ thống	8
2.1.1 Các tập hợp nền tảng (Sets)	8
2.1.2 Hệ thống tham số đầu vào (Parameters)	9
2.2 Hệ thống biến quyết định (Decision Variables)	10
2.2.1 Biến quyết định chính	11
2.2.2 Biến phụ tuyến tính hóa mục tiêu Công bằng (Minimax)	11
2.2.3 Biến phụ kiểm soát cặp ca trong ngày (Dynamic Inter-shift Variables)	12
2.3 Ràng buộc cứng (Core Hard Constraints)	13
2.3.1 Ràng buộc về nhu cầu nhân lực tối thiểu của ca thi (Capacity Constraint)	13
2.3.2 Ràng buộc chống trùng lịch và giới hạn vai trò (No Double-Booking Constraint)	13
2.3.3 Luật Chốt chặn chuyên môn (Lead Proctor Rule)	14
2.3.4 Ràng buộc trạng thái bận tuyệt đối (Hard Availability Constraint)	14
2.3.5 Ràng buộc chuyên môn lý tưởng (Ideal Qualification Constraint)	15
2.3.6 Ràng buộc di chuyển bất khả thi (Physical Feasibility Constraint)	15
2.4 Ràng buộc mềm - Ràng buộc kích hoạt điểm phạt vận hành (Operational Penalty Activation Constraints)	15

2.4.1	Ràng buộc kích hoạt phạt di chuyển khác cơ sở khả thi và phạt chờ đợi giữa các ca (Campus Travel and Waiting Penalty)	16
2.4.2	Ràng buộc kích hoạt phạt mệt mỏi (Fatigue Penalty Activation)	16
2.4.3	Kích hoạt điểm phạt tĩnh (Static Penalty Activation)	16
2.5	Hàm mục tiêu	17
2.5.1	Đối với mục tiêu công bằng (F_1)	17
2.5.2	Đối với mục tiêu phù hợp (F_2)	17
2.5.3	Hàm mục tiêu tổng hợp (Z)	18
3	TRIỂN KHAI MÃ NGUỒN VÀ BỘ GIẢI (IMPLEMENTATION)	18
3.1	Sự thay đổi khi triển khai từ lý thuyết	18
3.1.1	Biến nối lỏng xử lý nghịch lý hệ thống (Slack Variables)	19
3.1.2	Nối lỏng ràng buộc về nhân lực	19
3.1.3	Nối lỏng ràng buộc về trạng thái bận	20
3.1.4	Nối lỏng ràng buộc về năng lực cán bộ	20
3.2	Tiền xử lý dữ liệu	22
3.3	Cấu trúc chương trình và Bộ giải	23
3.3.1	Mã giả toàn diện hệ thống (System Orchestration)	23
3.3.2	data_preprocessing.py (Tiền xử lý dữ liệu)	25
3.3.3	model_builder.py (Xây dựng mô hình toán học)	26
3.3.4	solver.py (Giao tiếp bộ giải và Xuất kết quả)	27
3.3.5	Các kỹ thuật lập trình thực tiễn	28
4	Phân tích hiệu năng	29
4.1	Phân tích độ phức tạp	29
4.1.1	Phân tích độ phức tạp thời gian	30
4.1.2	Phân tích độ phức tạp không gian	33
4.1.3	Nhận xét	33
4.2	Đánh giá thời gian thực thi	33

4.3	So sánh với Baseline	35
4.4	Tỷ lệ Feasible/Infeasible theo Busy Rate	40
4.4.1	Phân tích và nhận xét	41
4.5	Khảo sát tập tối ưu Pareto và xác định hệ số phạt	42
4.5.1	Lý do khảo sát	42
4.5.2	Tập tối ưu Pareto là gì?	43
4.5.3	Phương pháp khảo sát	43
4.5.4	Kết quả khảo sát	44
4.5.5	Phân tích và nhận xét	45
5	Đề xuất mô hình mở rộng trong tương lai (Future Extension)	46
5.1	Giải pháp sử dụng Thuật toán Di truyền (Genetic Algorithm - GA) . . .	46
5.2	Giải pháp Tìm kiếm Lân cận (Local Search) và Luyện kim nhân tạo (Simulated Annealing - SA)	47
5.3	Phương pháp kết hợp Matheuristic (ILP + Heuristic)	47
5.4	Tối ưu hóa Hàm phạt thời gian chờ (Waiting Penalty Function)	48
6	Tổng kết và Bài học kinh nghiệm	49
6.1	Sự trưởng thành trong tư duy phân hoạch ràng buộc	49
6.2	Bài học về kiến trúc hệ thống và kiểm toán dữ liệu tĩnh	50
6.3	Kết luận về mặt năng lực thu nhận	50
A	Biên bản làm việc nhóm	51
	Tài liệu tham khảo	53

DANH MỤC HÌNH VẼ

4.1	Biểu đồ thời gian thực thi CBC Solver theo kích thước bài toán	34
4.2	So sánh chất lượng lịch phân công – Baseline vs. PuLP Solver	38
4.3	Phân phối khối lượng công việc – Baseline vs. PuLP Solver	38
4.4	So sánh điểm phạt: Baseline vs. PuLP Solver (CBC, Optimal)	39
4.5	Phân tích vi phạm ràng buộc cứng: Baseline vs. Solver	39
4.6	Biểu đồ Pareto	45

DANH MỤC BẢNG BIỂU

0.1	BẢNG PHÂN CÔNG VÀ ĐÁNH GIÁ TIẾN ĐỘ CÔNG VIỆC	1
2.4	Phân tích các tập hợp trong bài toán IAP	8
2.5	Bảng phân tích các tham số đầu vào của mô hình IAP	9
2.5	Bảng phân tích các tham số đầu vào của mô hình IAP	10
2.7	Biến quyết định chính của mô hình IAP	11
2.9	Biến phụ tuyến tính hóa hàm mục tiêu Minimax	12
3.11	Hệ thống biến nối lỏng (Slack) phục vụ quá trình Tuning	19
4.13	Ký hiệu và giá trị thực tế của bài toán	30
4.15	Độ phức tạp Giai đoạn 1 – Tiền xử lý dữ liệu	31
4.17	Độ phức tạp Giai đoạn 2 – Xây dựng mô hình	32
4.19	Độ phức tạp không gian	33
4.21	Thời gian thực thi CBC theo kích thước subset (hàng xám = dataset đầy đủ)	34
4.23	Tám tiêu chí so sánh Baseline vs. PuLP Solver	35
4.25	Kết quả so sánh định lượng Baseline vs. PuLP Solver (CBC, Optimal)	37
4.27	Kết quả thí nghiệm đo tỷ lệ Feasible/Infeasible theo hard_busy_rate	41
4.29	Kết quả Grid Search Pareto theo θ (cố định $\omega = 1,0$)	44
4.30	Bộ tham số được đề xuất sau khảo sát Pareto	46

1 Giới thiệu về bài toán phân công giám thị (IAP)

1.1 Tổng quan bài toán

Trong môi trường đại học, việc tổ chức thi cuối kì không chỉ đòi hỏi lập lịch phòng thi và ca thi hợp lý, mà còn phải phân công đúng số lượng cán bộ coi thi (CBCT) có đủ năng lực cho từng ca thi một cách chính xác. Bài toán này, được gọi là Bài Toán Phân Công Cán Bộ Coi Thi (Invigilator Assignment Problem - IAP), là một trong những nhiệm vụ hành chính phức tạp và tốn nhiều công sức nhất cho mỗi học kì tại Trường Đại học Bách Khoa TP.HCM (HCMUT).

Về bản chất, IAP đặt ra câu hỏi : cho trước 1 tập hợp cán bộ và một tập hợp ca thi, làm thế nào để phân công cán bộ vào các ca thi sao cho toàn bộ quy trình nghiêm ngặt được tuân thủ và lịch phân công tổng thể là công bằng, hiệu quả nhất có thể ? Trên thực tế, người lập trình phải xử lý đồng thời hàng chục hay hàng trăm ca thi trải dài trên nhiều tuần, hàng chục cán bộ với lịch bận khác nhau, nhiều cơ sở học tập, nhiều vai trò coi thi khác nhau và các quy định nghiêm ngặt của nhà trường. Việc lập lịch thủ công không chỉ mất nhiều thời gian mà còn dễ xảy ra xung đột lịch hay phân bổ khối lượng công việc không đều giữa các cán bộ.

Từ góc độ lý thuyết, IAP thuộc lớp bài toán Constraint Satisfaction Problem và có độ phức tạp NP-Hard. Điều này có nghĩa khi quy mô bài toán tăng lên, số lượng lịch phân công có thể tăng theo lũy thừa, khiến việc kiểm tra lại toàn bộ các trường hợp là bất khả thi. Do vậy, mô hình hóa toán học và các kỹ thuật tối ưu hóa là công cụ thiết yếu để giải quyết bài toán này trong thực tế.

1.2 Dữ liệu thực tế

Báo cáo này giải quyết vấn đề IAP dựa trên bộ dữ liệu thực tế (Đã được ẩn danh) được khoa Khoa học và kỹ thuật máy tính cung cấp, bao gồm lịch phân công cán bộ coi thi trong học kì 252. Bộ dữ liệu chứa một lịch phân công có sẵn (Baseline) - tức là một lịch đã tồn tại trước đó cùng với toàn bộ thông tin liên quan bao gồm mã ca thi, thời gian, cơ sở và vai trò của cán bộ trong ca thi.

Dữ liệu ghi nhận các ca thi trên hai cơ sở : Cơ sở 1 tại Lý Thường Kiệt (LTK) và cơ sở 2 tại Dĩ An. Mỗi ca yêu cầu 1 số lượng cụ thể các cán bộ trong các vai trò sau nhận một trong các vai trò sau : Cán bộ coi thi (CBCT), Thư kí và trưởng hội đồng coi thi.

Dữ liệu thực tế lấy từ datasheet : [Dataset_Anonymized_Invigilator_Assignment_Problem](#)

Lịch phân công hiện có trong bộ dữ liệu được sử dụng làm baseline để đối chiếu. Mục tiêu của cả nhóm là trích xuất các tham số bài toán từ dữ liệu của datasheet này, xây dựng mô hình quy hoạch nguyên tuyến tính (ILP) chính xác và sử dụng solver để tạo ra một lịch phân công mới tối hơn baseline về tính công bằng và hiệu quả.

1.3 Mục tiêu của đề tài

Đề tài này hướng đến các mục tiêu cụ thể sau:

- Xây dựng mô hình ILP đầy đủ cho IAP, bao gồm tập hợp , tham số, biến quyết định, hàm mục tiêu và ràng buộc.
- Tiền xử lý bộ dữ liệu thực tế để trích xuất đầu vào có cấu trúc solver.
- Bổ sung dữ liệu vị trí cư trú giả lập, cho cán bộ để minh họa chi phí đi lại.
- Triển khai và chạy mô hình ILP bằng solver miễn phí để thu được lịch phân công tối ưu.
- Đánh giá lịch tối ưu so với Baseline bằng các chỉ số định lượng về công bằng và hiệu quả.
- Đề xuất hướng mở rộng mô hình trong tương lai.

1.4 Phạm vi và giả thuyết

Đề bài toán có phạm vi rõ ràng, nhóm áp dụng giả thuyết sau:

- Tập cán bộ và tập cố định và biết trước
- Mỗi cán bộ chỉ được phân công tối đa 1 vai trò trong 1 ca thi.
- Cán bộ đã xuất hiện trong baseline ở một ca thi nào đó được xem là bạn cho các ca thi chung giờ.
- Thông tin cư trú của cán bộ không có sẵn trong dữ liệu ẩn danh do đó được giả lập ngẫu nhiên với seed cố định để đảm bảo tính toán, tái hiện được.
- Bài toán được mô hình hóa dạng Binary ILP : biến quyết định $x_{ijr} = 1$ nếu cán bộ i được phân công ca j và vai trò r , bằng 0 khi ngược lại.

2 Mô hình hóa toán học

2.1 Phân tích tập hợp và tham số hệ thống

Để xây dựng mô hình Chương trình tuyến tính nguyên (ILP) một cách chặt chẽ, trước hết nhóm tiến hành định nghĩa toán học cho toàn bộ hệ thống tập hợp và các tham số đầu vào trích xuất từ dữ liệu thực tế.

2.1.1 Các tập hợp nền tảng (Sets)

Hệ thống chỉ số và không gian nghiệm của bài toán được xây dựng dựa trên cấu trúc 3 tập hợp cốt lõi sau:

Tên tập hợp	Kí hiệu	Mô tả chi tiết
Tập hợp cán bộ	CB (hoặc I)	Tập hợp tất cả cán bộ coi thi có thể điều động, định danh bằng mã cán bộ ($CBxxx$). $ CB $ là tổng số cán bộ.
Tập hợp ca thi	CT (hoặc j)	Tập hợp tất cả các ca thi cần phân công, định danh bằng chuỗi kết hợp "Ngày_Ca Cơ sở" nhằm phân biệt không gian thời gian. Số lượng ca thi là $ CT $.
Tập hợp vai trò	R	Tập hợp cố định các nhiệm vụ trực phòng thi: $R = \{ 'CBCT', 'Thuky', 'TruongHD' \}$. Số lượng phần tử $ R = 3$.
Tập hợp Cơ sở	K	Tập hợp các cơ sở tổ chức thi (Ví dụ: Cơ sở 1 - LTK, Cơ sở 2 - Dĩ An). Số lượng phần tử $ K = 2$.

Bảng 2.4 Phân tích các tập hợp trong bài toán IAP

Ngoài ra, để phục vụ cho việc thiết lập các hệ ràng buộc logic phức tạp cũng như tính toán chính xác các cấu trúc điểm phạt động trong hàm mục tiêu, nhóm nghiên cứu đã xây dựng và định nghĩa thêm một số tập hợp phụ (Auxiliary/Derived Sets) từ nền tảng dữ liệu gốc:

- **Tập hợp các cặp ca thi cùng ngày không chồng lấn $\mathcal{A}$:** Định nghĩa tất cả các cặp ca thi (j_1, j_2) diễn ra trong cùng một ngày thi và tuân thủ thứ tự thời gian tuyến tính ($end_{j_1} \leq start_{j_2}$). Tập hợp này đóng vai trò làm không gian nền để cấu hình biến chỉ thị cặp ca, phục vụ tính điểm phạt di chuyển và lãng phí thời gian chờ đợi liên ca.

- **Tập hợp các cặp ca thi di chuyển bất khả thi $\mathcal{T}$:** Là một tập con đặc biệt của $\mathcal{A}$ ($\mathcal{T} \subset \mathcal{A}$), bao gồm các cặp ca thi diễn ra tại hai cơ sở khác nhau ($CS_{j_1} \neq CS_{j_2}$) nhưng có khoảng trống thời gian dịch chuyển giữa ca trước và ca sau nhỏ hơn 2 giờ ($start_{j_2} - end_{j_1} < 2.0$), gây ra sự bất khả thi vật lý tuyệt đối trong điều phối.
- **Tập hợp các cặp ca thi xung đột thời gian O :** Gồm nhóm các cặp ca thi diễn ra song song (chồng lấn thời gian) hoặc có thời điểm kết thúc và bắt đầu trùng khít nhau không khoảng đệm hành chính. Tập hợp này phục vụ trực tiếp cho ràng buộc cứng chống trùng lịch tuyệt đối (*No Double-Booking*).
- **Tập hợp các ngày thi D :** Tập hợp tất cả các ngày thi độc lập có phát sinh lịch tổ chức thi được trích xuất từ bộ dữ liệu Excel, cấu thành trục tọa độ thời gian để thiết lập cơ chế đếm số ca trực trong ngày nhằm kích hoạt cấu trúc phạt một mỗi lũy tiến của cán bộ.

2.1.2 Hệ thống tham số đầu vào (Parameters)

Các tham số cấu hình trạng thái, năng lực nhân sự, yêu cầu vận hành và trọng số phạt được định nghĩa chi tiết trong Bảng 2.5.

Bảng 2.5 Bảng phân tích các tham số đầu vào của mô hình IAP

Tên tham số	Kí hiệu	Kiểu dữ liệu	Ý nghĩa toán học và phân lớp dữ liệu
Tham số trạng thái bận	B_{ij}	Integer	Trạng thái khả dụng của cán bộ i tại ca j : • 0: Sẵn sàng (Available) • 1: Bận nhẹ (Soft-Busy, cho phép nối lỏng kèm phạt) • 2: Bận tuyệt đối (Hard-Busy, cấm phân công)
Nhu cầu nhân sự phòng thi	Cap_{jr}	Integer	Số lượng cán bộ tối thiểu bắt buộc phải có tại ca thi j tương ứng với vai trò $r \in R$.

Bảng 2.5 Bảng phân tích các tham số đầu vào của mô hình IAP

Tên tham số	Kí hiệu	Kiểu dữ liệu	Ý nghĩa toán học và phân lớp dữ liệu
Cấp bậc năng lực chuyên môn	L_i	Integer	Phân cấp trình độ của cán bộ i : • 1: Đủ điều kiện làm Giám thị (CBCT). • 2: Đủ điều kiện làm Thư ký và Giám thị. • 3: Mức cao nhất, đủ năng lực làm Trưởng hội đồng và tất cả vai trò dưới.
Tham số thời gian ca thi	$Date_j, Start_j, End_j$	Date / Float	Xác định ngày diễn ra ca thi j , thời điểm bắt đầu và thời điểm kết thúc (đổi sang đơn vị giờ dạng số thực để tính toán khoảng cách ca).
Địa điểm tổ chức ca thi	$Campus_j$	Categorical	Cơ sở diễn ra ca thi j (ví dụ: Cơ sở 1, Cơ sở 2) dùng để kiểm soát lỗi hỏng vật lý khi di chuyển.
Mức độ yêu thích cơ sở	$Campus_like_{ik}$	Integer	Điểm số thể hiện mức độ ưu tiên chọn cơ sở k của cán bộ i , nhận giá trị trong tập $\{1, 2, 3\}$.
Trọng số cân bằng hàm mục tiêu	ω, θ	Real	• ω: Trọng số áp cho tổng chi phí phạt nguyên vọng tĩnh, động và phạt Slack. • θ: Trọng số áp cho hàm mục tiêu công bằng (Fairness).
Hằng số thuế phạt nói lỏng	$TAX_{lack}, TAX_{busy}, TAX_{qual}$	Real	Các mức phạt tài chính cực lớn áp cho các biến nói lỏng nhằm cứu vãn nghiệm khi bài toán rơi vào trạng thái bất khả thi (Infeasible).

2.2 Hệ thống biến quyết định (Decision Variables)

Để chuyển đổi bài toán lập lịch trực thi thực tế thành một mô hình Quy hoạch tuyến tính nguyên (ILP) có khả năng tự chẩn đoán và nói lỏng, hệ thống biến được nhóm

thiết kế bao gồm: biến quyết định chính, các biến nối lỏng (Slack variables) phục vụ chiến lược cứu vãn nghiệm, và các biến phụ tuyến tính hóa (Auxiliary variables).

2.2.1 Biến quyết định chính

Biến quyết định cốt lõi của mô hình là biến nhị phân đại diện cho trạng thái phân công công việc của từng cán bộ:

Ký hiệu	Miền	Kiểu	Ý nghĩa toán học
X_{ijr}	$\{0, 1\}$	Binary	Nhận giá trị 1 nếu cán bộ $i \in CB$ được phân công vào ca thi $j \in CT$ với vai trò cụ thể $r \in R$; nhận giá trị 0 nếu không phân công.

Bảng 2.7 Biến quyết định chính của mô hình IAP

Biến X_{ijr} được xác định trên toàn bộ không gian tích Descartes của các tập hợp nền tảng:

$$X_{ijr} \in \{0, 1\}, \quad \forall i \in CB, \forall j \in CT, \forall r \in R \quad (2.1)$$

Với quy mô dữ liệu thử nghiệm gồm khoảng 60 cán bộ, hơn 100 ca thi và 3 vai trò trực, mô hình sản sinh xấp xỉ **18.000 biến nhị phân**. Ràng buộc độc nhất vai trò của một cán bộ trong một ca thi cụ thể được mô tả bởi bất đẳng thức:

$$\sum_{r \in R} X_{ijr} \leq 1, \quad \forall i \in CB, \forall j \in CT \quad (2.2)$$

2.2.2 Biến phụ tuyến tính hóa mục tiêu Công bằng (Minimax)

Để thực hiện chiến lược cào bằng khối lượng công việc, mô hình đo lường khoảng cách cực trị công việc thông qua kỹ thuật Minimax với các biến phụ sau:

Ký hiệu	Miền	Kiểu	Ý nghĩa toán học
W_i	$\mathbb{Z}^+$	Integer	Tổng số ca thi thực tế mà cán bộ i phải đảm nhiệm: $W_i = \sum_{j \in CT} \sum_{r \in R} X_{ijr}$.
W_{high}	$\mathbb{Z}^+$	Integer	Chặn trên tuyến tính của khối lượng công việc: $W_{\text{high}} \geq W_i, \forall i \in CB$.
W_{low}	$\mathbb{Z}^+$	Integer	Chặn dưới tuyến tính của khối lượng công việc: $W_{\text{low}} \leq W_i, \forall i \in CB$.

Bảng 2.9 Biến phụ tuyến tính hóa hàm mục tiêu Minimax

2.2.3 Biến phụ kiểm soát cặp ca trong ngày (Dynamic Inter-shift Variables)

Để tối ưu hóa sức khỏe cán bộ và triệt tiêu các lịch trình bất hợp lý phát sinh trong ngày, mô hình cấu hình hai nhóm biến phụ động nhị phân:

- Biến cặp ca trùng/phạt di chuyển ($Y_{i,j_1,j_2}^{\text{pair}}$):** Xét tập hợp tất cả các cặp ca thi (j_1, j_2) diễn ra trong cùng một ngày là $\mathcal{A}$ và không bị xung đột về mặt thời gian thỏa $\text{end}_{j_1} < \text{start}_{j_2}$. Vì nhóm tiền xử lý dữ liệu phát hiện có sự giao thoa quỹ thời gian nghỉ hoặc cần chế tài hình phạt (di chuyển giữa 2 cơ sở, thời gian chờ đợi quá dài). Biến nhị phân được định nghĩa:

$$Y_{i,j_1,j_2}^{\text{pair}} \in \{0, 1\}, \quad \forall i \in CB, \forall (j_1, j_2) \in \mathcal{A} \quad (2.3)$$

Biến này được liên kết tuyến tính với biến quyết định chính thông qua ràng buộc logic:

$$Y_{i,j_1,j_2}^{\text{pair}} \geq \sum_{r \in R} X_{ij_1r} + \sum_{r \in R} X_{ij_2r} - 1 \quad (2.4)$$

- Biến mệt mỏi bậc thang lũy tiến ($Y_{i,d,k}^{\text{fatigue}}$):** Để phạt nặng các trường hợp ép một cán bộ trực quá nhiều ca trong cùng một ngày $d \in D$ (D là tập các ngày thi), mô hình sử dụng cấu trúc phạt bậc thang tăng dần với $k \in \{3, 4, 5\}$ tương ứng với ca làm việc thứ k trong ngày:

$$Y_{i,d,k}^{\text{fatigue}} \in \{0, 1\}, \quad \forall i \in CB, \forall d \in D, k \in \{3, 4, 5\} \quad (2.5)$$

Mối quan hệ kích hoạt cấu trúc hình phạt tuân thủ hệ bất đẳng thức logic:

$$\sum_{j \in \text{shifts_by_day}[d]} \sum_{r \in R} X_{ijr} \leq 2 + Y_{i,d,3}^{\text{fatigue}} + Y_{i,d,4}^{\text{fatigue}} + Y_{i,d,5}^{\text{fatigue}} \quad (2.6)$$

$$Y_{i,d,4}^{\text{fatigue}} \leq Y_{i,d,3}^{\text{fatigue}} \quad (2.7)$$

$$Y_{i,d,5}^{\text{fatigue}} \leq Y_{i,d,4}^{\text{fatigue}} \quad (2.8)$$

2.3 Ràng buộc cứng (Core Hard Constraints)

Các ràng buộc cứng đại diện cho các quy tắc vận hành bắt buộc của nhà trường và phòng đào tạo. Trong mô hình lý tưởng, mọi phương án phân công hợp lệ đều không được phép vi phạm bất kỳ điều kiện nào dưới đây. Các ràng buộc này hình thành nên tầng khả thi cốt lõi (*core feasibility layer*) của toàn bộ hệ thống tối ưu.

2.3.1 Ràng buộc về nhu cầu nhân lực tối thiểu của ca thi (Capacity Constraint)

Để đảm bảo mỗi ca thi được tổ chức đầy đủ nhân sự theo đúng quy mô vận hành, với mỗi ca thi j và vai trò chuyên môn r , tổng số cán bộ được phân công phải bằng đúng nhu cầu yêu cầu Cap_{jr} được suy diễn từ dữ liệu baseline:

$$\sum_{i \in CB} x_{ijr} = Cap_{jr}, \quad \forall j \in CT, \forall r \in R \quad (2.9)$$

Ràng buộc này đảm bảo toàn bộ vị trí công tác trong từng ca thi đều được đáp ứng đầy đủ về mặt số lượng nhân sự.

Ngoài ra, đối với các vai trò có nhu cầu thực sự ($Cap_{jr} > 0$), hệ thống bổ sung thêm một điều kiện bảo toàn vận hành nhằm đảm bảo luôn tồn tại ít nhất một cán bộ đủ chuẩn chuyên môn đảm nhiệm vai trò tương ứng:

$$\sum_{i \in Q_{jr}} x_{ijr} \geq 1, \quad \forall j \in CT, \forall r \in R \text{ mà } Cap_{jr} > 0 \quad (2.10)$$

Trong đó:

$$Q_{jr} = \{i \in CB \mid (L_i \geq L_r^{req}) \wedge (B_{ij} \neq 2)\} \quad (2.11)$$

là tập các cán bộ có trình độ chuyên môn phù hợp với vai trò r .

Ngược lại, nếu một vai trò không tồn tại nhu cầu nhân lực:

$$x_{ijr} = 0, \quad \forall i \in CB, \forall (j, r) : Cap_{jr} = 0 \quad (2.12)$$

Ràng buộc này giúp triệt tiêu hoàn toàn các phân công dư thừa hoặc sai mục đích.

2.3.2 Ràng buộc chống trùng lịch và giới hạn vai trò (No Double-Booking Constraint)

Một cán bộ không thể đồng thời thực hiện nhiều nhiệm vụ tại các ca thi có xung đột thời gian hoặc kiêm nhiệm nhiều vai trò trong cùng một ca thi.

Mô hình trước tiên xác định tập các cặp ca thi xung đột thời gian O dựa trên hai trường hợp thực tế:

- Trường hợp chồng lấn thời gian (Overlapping Shifts):

$$\max(\text{start}_{j_1}, \text{start}_{j_2}) < \min(\text{end}_{j_1}, \text{end}_{j_2}) \quad (2.13)$$

- Trường hợp ca liền kề không có khoảng đệm (Back-to-Back Shifts):

$$(\text{end}_{j_1} = \text{start}_{j_2}) \vee (\text{end}_{j_2} = \text{start}_{j_1}) \quad (2.14)$$

Nếu cặp ca (j_1, j_2) thỏa một trong hai điều kiện trên thì:

$$(j_1, j_2) \in O \quad (2.15)$$

Khi đó, mô hình áp đặt ràng buộc chống trùng lịch:

$$\sum_{r \in R} x_{ij_1r} + \sum_{r \in R} x_{ij_2r} \leq 1, \quad \forall i \in CB, \forall (j_1, j_2) \in O \quad (2.16)$$

Ngoài ra, trong cùng một ca thi, một cán bộ không được kiêm nhiệm nhiều vai trò:

$$\sum_{r \in R} x_{ijr} \leq 1, \quad \forall i \in CB, \forall j \in CT \quad (2.17)$$

2.3.3 Luật Chốt chặn chuyên môn (Lead Proctor Rule)

Nhằm đảm bảo an toàn pháp lý trên thực tế, hệ thống thiết lập thêm một chốt chặn bắt buộc phải có ít nhất một nhân sự điều phối đạt chuẩn cho các vị trí có nhu cầu lớn hơn 0. Gọi $Qualified(j, r) = \{i \in CB \mid L_i \geq ReqLevel(r) \text{ và } B_{ij} \neq 2\}$:

$$\sum_{i \in Qualified(j, r)} x_{ijr} \geq 1, \quad \forall j \in CT, \forall r \in R \text{ (nếu } Cap_{jr} > 0) \quad (2.18)$$

Sự hiện diện của ít nhất một chuyên gia đạt chuẩn là chốt chặn cuối cùng để xử lý các sự cố sự phạm phát sinh trong phòng thi.

2.3.4 Ràng buộc trạng thái bận tuyệt đối (Hard Availability Constraint)

Nếu cán bộ i được đánh dấu ở trạng thái bận tuyệt đối tại ca thi j ($B_{ij} = 2$), hệ thống không cho phép phát sinh bất kỳ phân công nào:

$$\sum_{r \in R} x_{ijr} = 0, \quad \forall i \in CB, \forall j \in CT : B_{ij} = 2 \quad (2.19)$$

Các trường hợp bận mềm ($B_{ij} = 1$) không bị xem là bất khả thi tuyệt đối và sẽ được xử lý trong tầng nói lỏng thực tế (*feasibility recovery layer*) ở phần sau.

2.3.5 Ràng buộc chuyên môn lý tưởng (Ideal Qualification Constraint)

Mỗi cán bộ có trình độ chuyên môn L_i nhất định để đảm nhận các vai trò khác nhau:

- $L_i = 1$: chỉ được làm CBCT.
- $L_i = 2$: được làm CBCT hoặc Thư ký.
- $L_i = 3$: được làm mọi vai trò.

Trong mô hình lý tưởng, cán bộ không được phép đảm nhận vai trò vượt quá năng lực chuyên môn:

$$x_{ij, \text{Thuky}} = 0, \quad \forall i \in CB, \forall j \in CT : L_i < 2 \quad (2.20)$$

$$x_{ij, \text{TruongHD}} = 0, \quad \forall i \in CB, \forall j \in CT : L_i < 3 \quad (2.21)$$

Các trường hợp nối lỏng chuyên môn sẽ được mô hình xử lý riêng trong tầng phục hồi tính khả thi thực tế.

2.3.6 Ràng buộc di chuyển bất khả thi (Physical Feasibility Constraint)

Nhằm đảm bảo cán bộ có đủ thời gian di chuyển giữa các cơ sở, mô hình xây dựng tập các cặp ca thi có xung đột di chuyển:

$$\mathcal{T} = \{(j_1, j_2) \in \mathcal{A} \mid CS_{j_1} \neq CS_{j_2} \wedge (\text{start}_{j_2} - \text{end}_{j_1}) < 2\} \quad (2.22)$$

Khi đó:

$$\sum_{r \in R} x_{i, j_1, r} + \sum_{r \in R} x_{i, j_2, r} \leq 1, \quad \forall i \in CB, \forall (j_1, j_2) \in \mathcal{T} \quad (2.23)$$

Ràng buộc này đảm bảo không phát sinh các lịch phân công bất khả thi về mặt vật lý giữa các cơ sở thi khác nhau.

2.4 Ràng buộc mềm - Ràng buộc kích hoạt điểm phạt vận hành (Operational Penalty Activation Constraints)

Các ràng buộc trong mục này không đại diện cho điều kiện bất khả thi tuyệt đối, mà đóng vai trò kích hoạt các biến cờ nhị phân phục vụ cho việc tính toán chi phí phạt động trong hàm mục tiêu.

2.4.1 Ràng buộc kích hoạt phạt di chuyển khác cơ sở khả thi và phạt chờ đợi giữa các ca (Campus Travel and Waiting Penalty)

Với mỗi cặp $(j_1, j_2) \in \mathcal{A}$, mô hình tính toán trước chi phí phạt:

$$P_{j_1 j_2}^{pair} = P_{j_1 j_2}^{travel} + P_{j_1 j_2}^{wait} \quad (2.24)$$

Trong đó $P_{j_1 j_2}^{travel}$ là điểm phạt khi mà di chuyển giữa các cơ sở với khoảng cách thời gian ($gap \geq 2$) và $P_{j_1 j_2}^{wait}$ là điểm phạt cho khoảng thời gian ($gap > 2$) cán bộ phải chờ đợi giữa 2 ca làm việc :

$$P_{j_1 j_2}^{travel} = \max(0, 50 - 10 \cdot (gap - 2)) \quad (2.25)$$

$$P_{j_1 j_2}^{wait} = 15 \cdot (gap - 2) \quad (2.26)$$

Để kích hoạt chi phí này khi cùng một cán bộ được gán vào cả hai ca:

$$Y_{ij_1 j_2}^{pair} \geq \sum_{r \in R} x_{ij_1 r} + \sum_{r \in R} x_{ij_2 r} - 1 \quad (2.27)$$

Biến nhị phân $Y_{ij_1 j_2}^{pair}$ đóng vai trò như một biến kích hoạt phạt động liên ca.

2.4.2 Ràng buộc kích hoạt phạt mệt mỏi (Fatigue Penalty Activation)

Mô hình khuyến khích mỗi cán bộ không nên làm quá 2 ca trong cùng một ngày. Khi số ca vượt quá ngưỡng này, hệ thống sẽ kích hoạt các biến cờ nhị phân phân cấp:

$$\sum_{j \in shifts_by_day[d]} x_{ijr} \leq 2 + Y_{i,d,3}^{fatigue} + Y_{i,d,4}^{fatigue} + Y_{i,d,5}^{fatigue} \quad (2.28)$$

Kèm theo quan hệ thứ bậc:

$$Y_{i,d,5}^{fatigue} \leq Y_{i,d,4}^{fatigue} \leq Y_{i,d,3}^{fatigue} \quad (2.29)$$

Các biến $Y^{fatigue}$ hoạt động như các biến kích hoạt phạt lũy tiến, giúp gia tăng chi phí khi cán bộ bị phân công quá tải trong cùng một ngày.

2.4.3 Kích hoạt điểm phạt tĩnh (Static Penalty Activation)

Bên cạnh các chi phí động liên ca, mô hình còn sử dụng ma trận điểm phạt tĩnh P_{ijr} để phản ánh:

$$P_{ijr} = penalty_campus + penalty_qual$$

Trong đó :

- **penalty_campus** là điểm phạt cho độ yêu thích về cơ sở của ca thi. Được tính bằng :

$$penalty_campus = 10 \cdot (1 - \frac{Campus_like_{i,CS_j}}{3})$$

- **penaty_qual** là đại diện cho điểm phạt lãng phí về năng lực. Với L_r^{req} đại diện cho yêu cầu năng lực của vai trò r. Trong mô hình lý tưởng, được tính bằng :

$$penalty_qual = 5 \cdot (L_i - L_r^{req})$$

Do P_{ijr} là hằng số được tính toán trước từ dữ liệu đầu vào, thành phần này được đưa trực tiếp vào hàm mục tiêu thông qua biến quyết định:

$$P_{ijr} \cdot x_{ijr} \quad (2.30)$$

Khi một phân công có mức độ không phù hợp cao, giá trị P_{ijr} lớn sẽ làm tăng tổng chi phí tối ưu, từ đó định hướng solver tìm kiếm các phương án phân công hợp lý hơn.

2.5 Hàm mục tiêu

Mục tiêu tối ưu hóa của mô hình IAP hướng đến việc giải quyết hài hòa hai bài toán cốt lõi: **sự phù hợp tối đa khi phân công** (giảm thiểu chi phí/một mỗi) và **sự công bằng tuyệt đối về khối lượng công việc**.

2.5.1 Đối với mục tiêu công bằng (F_1)

Để ngăn chặn triệt để tình trạng phân bổ không đồng đều tải trọng công việc (ví dụ: một cán bộ phải gác quá nhiều ca trong khi cán bộ khác lại hoàn toàn trống lịch), mô hình áp dụng hàm mục tiêu tối thiểu hóa khoảng cách cực đại thông qua biến phụ toán học Minimax:

$$\text{Minimize } F_1 = W_{\text{high}} - W_{\text{low}} \quad (2.31)$$

2.5.2 Đối với mục tiêu phù hợp (F_2)

Hàm mục tiêu F_2 chịu trách nhiệm tổng hợp toàn bộ các điểm phạt (Penalty) từ các thành phần tĩnh, động và cả các cấu trúc nối lỏng hệ thống. Dựa trên các hệ ràng buộc tuyến tính hóa đã thiết lập ở các chương trước, F_2 được phát biểu chính xác như

sau:

$$\begin{aligned} \text{Minimize } F_2 = & \underbrace{\sum_{i \in CB} \sum_{j \in CT} \sum_{r \in R} P_{ijr} \cdot x_{ijr}}_{\text{Penalty tính (Sở thích \& Lãng phí chuyên môn)}} + \underbrace{\sum_{i \in CB} \sum_{(j_1, j_2) \in \mathcal{A}} P_{j_1, j_2}^{\text{pair}} \cdot Y_{i, j_1, j_2}^{\text{pair}}}_{\text{Penalty di chuyển \& chờ đợi liên ca}} \\ & + \underbrace{\sum_{i \in CB} \sum_{d \in D} (40 \cdot Y_{i, d, 3}^{\text{fatigue}} + 80 \cdot Y_{i, d, 4}^{\text{fatigue}} + 150 \cdot Y_{i, d, 5}^{\text{fatigue}})}_{\text{Penalty mệt mỏi (Lũy tiến theo bậc ca trong ngày)}} \end{aligned} \quad (2.32)$$

2.5.3 Hàm mục tiêu tổng hợp (Z)

Hàm mục tiêu đa mục tiêu cuối cùng của mô hình được thiết lập thông qua phương pháp tổng tuyến tính có trọng số:

$$\text{Minimize } Z = \omega \cdot F_2 + \theta \cdot F_1 \quad (2.33)$$

Trong đó, ω và θ là các tham số trọng số do người vận hành cấu hình nhằm điều khiển thứ tự ưu tiên của hệ thống. Trong các kịch bản thực nghiệm thực tế tại HCMUT, nhóm nghiên cứu sẽ sử dụng bộ tham số $\omega = 1.0$ và $\theta = 20.0$ để đặt làm thử nghiệm cho bộ mô hình, sau các kết quả thống kê bộ tham số này sẽ thay đổi cho phù hợp với mô hình (điều này sẽ được đề cập ở phần sau).

3 TRIỂN KHAI MÃ NGUỒN VÀ BỘ GIẢI (IMPLEMENTATION)

3.1 Sự thay đổi khi triển khai từ lý thuyết

Khi chuyển đổi mô hình toán học lý thuyết sang mã nguồn vận hành thực tế, thách thức lớn nhất là sự xung đột hoặc thiếu hụt tài nguyên cục bộ trong dữ liệu thực nghiệm (như thiếu hụt nhân sự diện rộng hoặc lịch bận bất khả kháng của giảng viên). Nếu giữ nguyên tính tuyệt đối của các ràng buộc hành chính như thiết kế lý thuyết ban đầu, bộ giải (solver) sẽ lập tức báo lỗi vô nghiệm (Infeasible) và dừng chương trình. Để khắc phục triệt để "nút thắt cổ chai" này, nhóm nghiên cứu đã áp dụng chiến lược *Nới lỏng ràng buộc (Constraint Relaxation)*. Cụ thể, các ràng buộc quản lý hành chính "cứng" được mềm hóa thành các cấu trúc phòng ngự đàn hồi thông qua việc nhúng trực tiếp hệ thống biến bù (Slack variables) và các mức thuế phạt cực lớn (Big-M Penalty) vào tiến trình khởi tạo mô hình. Cơ chế tự chẩn đoán và phục hồi tính khả thi này được triển khai chi tiết qua các thành phần dưới đây:

3.1.1 Biến nói lỏng xử lý nghịch lý hệ thống (Slack Variables)

Đây là các biến vô cùng quan trọng đóng vai trò "mềm hóa" các ràng buộc cứng nhằm đảm bảo tính khả thi của không gian nghiệm (tránh lỗi Infeasible khi tài nguyên bị thiếu hụt trầm trọng).

Ký hiệu	Miền	Kiểu	Ý nghĩa và chức năng kỹ thuật
s_{jr}^{cap}	$\mathbb{Z}^+$	Integer	Đại diện cho số lượng nhân sự bị thiếu hụt thực tế tại ca thi j đối với vai trò r so với nhu cầu tối thiểu Cap_{jr} .
s_{ijr}^{busy}	$\{0, 1\}$	Binary	Hoạt động như cờ báo hiệu , nhận giá trị 1 khi mô hình bắt buộc điều động cán bộ i vào ca j dù có đăng ký bận nhẹ ($B_{ij} = 1$).
s_{ijr}^{qual}	$\{0, 1\}$	Binary	Hoạt động như cờ báo hiệu , nhận giá trị 1 khi cán bộ i bị ép phân công vào vai trò r vượt quá cấp bậc chuyên môn.

Bảng 3.11 Hệ thống biến nói lỏng (Slack) phục vụ quá trình Tuning

Các biến Slack trên đều được giới hạn ở miền không âm (≥ 0) và sẽ bị áp các mức thuế phạt cực lớn (TAX) trong cấu trúc hàm mục tiêu tối thiểu hóa chi phí vận hành.

3.1.2 Nói lỏng ràng buộc về nhân lực

Trong thực tế, do sự thiếu hụt nhân sự diện rộng hoặc do trùng lặp lịch biểu, số lượng cán bộ khả dụng có thể không đáp ứng đủ nhu cầu Cap_{jr} của ca thi. Để tránh lỗi vô nghiệm, hệ thống cho phép ca thi được vận hành trong tình trạng thiếu người, phần thiếu hụt này sẽ được hấp thụ bởi biến bù s_{jr}^{cap} . Ràng buộc (1) được viết lại như sau:

$$\sum_{i \in CB} x_{ijr} + s_{jr}^{cap} = Cap_{jr}, \quad \forall j \in CT, \forall r \in R \quad (3.34)$$

Với cơ chế này, nếu đủ người, s_{jr}^{cap} sẽ tự động bằng 0. Nếu thiếu người, s_{jr}^{cap} sẽ mang giá trị dương đại diện cho số người bị thiếu và ngay lập tức kích hoạt mức thuế phạt lớn (10000/một người thiếu) trong hàm mục tiêu, buộc thuật toán (solver) phải nỗ lực tối đa để lấp đầy các vị trí trước khi chấp nhận bỏ trống.

Lưu ý hệ thống: Cơ chế nói lỏng bằng biến s_{jr}^{cap} chỉ được áp dụng cho phương trình định biên số lượng cán bộ. Ngược lại, luật chốt chặn chất lượng phòng thi *Lead*

Proctor Rule (Phương trình 2.18) vẫn được giữ nguyên làm ràng buộc cứng tuyệt đối không chứa biến nối lỏng. Điều này đảm bảo tính an toàn hành chính tối cao cho kỳ thi, chấp nhận để hệ thống báo vô nghiệm nhằm cảnh báo ban giáo vụ nếu một ca thi hoàn toàn cạn kiệt nhân sự cốt lõi đạt chuẩn, thay vì tự động tạo ra một lịch phân công sai phạm pháp lý.

3.1.3 Nối lỏng ràng buộc về trạng thái bận

Cũng dựa trên tình trạng thiếu hụt nhân sự đã nêu, khi không còn cán bộ nào rảnh rỗi, hệ thống bắt buộc phải huy động các cán bộ đã đăng ký trạng thái bận nhẹ ($B_{ij} = 1$). Để kiểm soát tình trạng này, mô hình áp đặt một ràng buộc liên kết giữa quyết định phân công và cờ báo hiệu bận:

$$x_{ijr} \leq s_{ijr}^{\text{busy}}, \quad \forall i \in CB, \forall j \in CT, \forall r \in R \text{ mà } B_{ij} = 1 \quad (3.35)$$

Giải thích cơ chế:

- Nếu cán bộ không bị phân công ($x_{ijr} = 0$), hàm mục tiêu tối thiểu hóa (Minimization) sẽ tự động ép biến s_{ijr}^{busy} về 0 để tránh bị phạt.
- Nếu cán bộ bị ép phân công ($x_{ijr} = 1$), bất phương trình buộc s_{ijr}^{busy} phải lớn hơn hoặc bằng 1. Do đây là biến nhị phân, nó bắt buộc mang giá trị 1, từ đó đánh thức khoản phạt (5000) trong hàm F_2 .

3.1.4 Nối lỏng ràng buộc về năng lực cán bộ

Trong trường hợp cạn kiệt nhân sự có chuyên môn cao tại một số khung giờ cao điểm, để đảm bảo ca thi vẫn có người vận hành, hệ thống buộc phải nối lỏng giới hạn vai trò, cho phép cán bộ đảm nhận vị trí vượt quá cấp bậc hiện tại ($L_i < L_r^{\text{req}}$). Ràng buộc về chuyên môn được tổng quát hóa và mềm hóa thành điều kiện kích hoạt cờ nhị phân:

$$x_{ijr} \leq s_{ijr}^{\text{qual}}, \quad \forall i \in CB, \forall j \in CT, \forall r \in R \text{ mà } L_i < L_r^{\text{req}} \quad (3.36)$$

Mỗi khi một cán bộ bị phân công sai ngạch chuyên môn ($x_{ijr} = 1$), biến Slack nhị phân s_{ijr}^{qual} sẽ bị cưỡng ép bật lên 1 và chịu mức thuế phạt hành chính cố định rất lớn là $TAX_{\text{qual}} = 5000$.

Trong mô hình lý tưởng (mục 2.4.3), thành phần penalty_qual chỉ xét hành vi sắp ca "lãng phí năng lực" vì trường hợp vượt cấp bị cấm tuyệt đối bởi bởi ràng buộc năng lực cán bộ. Tuy nhiên, khi áp dụng chiến lược nối lỏng ràng buộc để đảm bảo tính

khả thi của nghiệm, cán bộ có thể được phân công vượt cấp trong tình huống cạn kiệt nhân sự. Để định hướng solver tránh những phân công này và ưu tiên chọn người thiếu ít bậc nhất, $penalty_qual$ được mở rộng thành hàm hai thành phần bất đối xứng::

$$Penalty_{qual}(i, r) = 5 \cdot \max(0, L_i - L_r^{req}) + 50 \cdot \max(0, L_r^{req} - L_i) \quad (3.37)$$

Hàm toán học này thiết lập cơ chế tài chia làm hai kịch bản độc lập:

- **Trạng thái lãng phí năng lực (Overqualification - $L_i > L_r^{req}$):** Thành phần thứ hai bằng 0, thành phần thứ nhất kích hoạt mức phạt tuyến tính nhẹ (5 điểm cho mỗi bậc chênh lệch). Điều này giúp hệ thống hạn chế việc điều động các cán bộ cấp cao (như Trưởng hội đồng) đi làm các công việc phổ thông (như Giám thị phòng thi) khi nguồn lực xã hội còn dư dả.
- **Trạng thái thiếu hụt năng lực (Underqualification - $L_i < L_r^{req}$):** Thành phần thứ nhất bằng 0, thành phần thứ hai kích hoạt mức phạt vi phạm chuyên môn (50 điểm cho mỗi bậc thiếu hụt). Kết hợp với mức thuế phạt cố định 5000 điểm của biến Slack s_{ijr}^{qual} , hệ thống hình thành cơ chế phạt hai lớp (Double-layer Penalty Scheme). Cơ chế này đảm bảo Solver luôn ưu tiên chọn người thiếu ít bậc nhất (ví dụ: chọn cán bộ có $L_i = 2$ gánh vai trò $L_r^{req} = 3$ thay vì chọn người có $L_i = 1$) trong kịch bản bắt buộc phải nối lỏng hệ thống.

Từ các thay đổi trên hàm F2 mới tức là hàm tối ưu về điểm phạt sẽ được bổ sung như sau :

$$\begin{aligned} \text{Minimize } F_2 = & \underbrace{\sum_{i \in CB} \sum_{j \in CT} \sum_{r \in R} P_{ijr} \cdot x_{ijr}}_{\text{Penalty tính (Sở thích \& Lãng phí chuyên môn)}} + \underbrace{\sum_{i \in CB} \sum_{(j_1, j_2) \in \mathcal{A}} P_{j_1, j_2}^{\text{pair}} \cdot Y_{i, j_1, j_2}^{\text{pair}}}_{\text{Penalty di chuyển \& chờ đợi liên ca}} \\ & + \underbrace{\sum_{i \in CB} \sum_{d \in D} (40 \cdot Y_{i, d, 3}^{\text{fatigue}} + 80 \cdot Y_{i, d, 4}^{\text{fatigue}} + 150 \cdot Y_{i, d, 5}^{\text{fatigue}})}_{\text{Penalty mệt mỏi (Lũy tiến theo bậc ca trong ngày)}} \\ & + \underbrace{10000 \sum_{j \in CT} \sum_{r \in R} s_{j, r}^{\text{cap}} + 5000 \sum_{i \in CB} \sum_{j \in CT} \sum_{r \in R} s_{i, j, r}^{\text{busy}} + 5000 \sum_{i \in CB} \sum_{j \in CT} \sum_{r \in R} s_{i, j, r}^{\text{qual}}}_{\text{Thuế phạt biến Slack (Nối lỏng ràng buộc cứng)}} \end{aligned} \quad (3.38)$$

Nhận xét hệ thống: Trong kiến trúc này, các biến s^{cap} , s^{busy} , và s^{qual} mang giá trị không âm và chỉ bị kích hoạt khi không gian dữ liệu đầu vào thực sự bế tắc, buộc thuật toán phải hy sinh ràng buộc hành chính để tìm ra phương án. Thay vì tiếp cận theo hướng "chạy đến khi báo lỗi vô nghiệm (Infeasible) rồi mới dùng thuật toán rà soát để nối lỏng", việc nhúng trực tiếp cơ chế này vào mô hình ngay từ đầu mang lại hai ưu điểm vượt trội:

- **Tối ưu hiệu năng thực thi:** Triệt tiêu hoàn toàn chi phí thời gian (overhead) của việc phải khởi tạo và gọi bộ giải (solver) chạy lại nhiều lần.
- **Cung cấp phương án "Tiệm cận tốt nhất" :** Hệ thống đảm bảo luôn trả về một lịch phân công hoàn chỉnh. Người dùng có thể dựa trực tiếp vào các khoản thuế phạt và giá trị của biến Slack trên báo cáo để nhận diện chính xác các "điểm nghẽn" nhân sự, từ đó đưa ra quyết định can thiệp thủ công (chỉnh sửa lịch) một cách tinh gọn và minh bạch.

Từ tất cả những gì đã nêu và mở rộng ở các phần trên, nhóm quyết định **các ràng buộc cùng hàm F2 mới** sẽ được ứng dụng vào **mô hình thực tế** mà nhóm hiện thực trên code, dựa trên tiêu chí tối ưu và hạn chế vấn đề tồn đọng của bản thân mô hình lý thuyết lý tưởng mà đã đề ra ban đầu để tìm ra các nghiệm tối ưu cho các lần giải.

3.2 Tiền xử lý dữ liệu

Việc tiền xử lý dữ liệu và chuẩn hóa là một bước vô cùng cần thiết trong việc đọc dữ liệu từ data sheet cho trước. Ta cần trích, tách, chuẩn hóa các dữ liệu cần cho mô hình như sau :

- **Đối với tập Cán bộ :** Lấy trực tiếp từ cột Mã số cán bộ
- **Đối với tập Ca thi :** Do có trường hợp ca thi sẽ trùng nhau nhưng khác cơ sở nên ta sẽ chuẩn hóa dữ liệu theo dạng "Mã ca thi | cơ sở " .
- **Đối với tập vai trò :** ta thực hiện việc chuẩn hóa lấy các vai trò sau dấu gạch nối "_" trong cột "Nhiệm vụ".
- **Đối với cơ sở của ca thi J :** ta trích trực tiếp từ cột cơ sở. Tuy nhiên data lại có sự thiếu sót ở các cột cuối do vậy nhóm thực hiện chuẩn hóa thêm lấy data từ cột Nhiệm vụ nếu cột cơ sở bị khuyết, nếu là LTK thì chuyển thành "Cơ sở 1", Dian thì thành "Cơ sở 2".
- **Thời gian :** bao gồm ngày lấy từ việc chuẩn hóa cột Ngày (bỏ đi các số 0 phía sau) và giờ lấy từ việc chuẩn hóa cột "Giờ" (ví dụ 8g30 thành 8,5.)

Đối với các tham số cần tính toán:

- **Tham số bận của cán bộ :** ta sẽ cho sinh ngẫu nhiên trong lần chạy này vì không nắm rõ lịch của giảng viên, nhưng người dùng vẫn có thể nhập vào ở các lần chạy khác nếu có thông tin.
- **Năng lực của nhân viên :** Đối với tham số này ta sẽ lấy trực tiếp từ vai trò của cán bộ đó từ data sheet (hoặc nếu không có thông tin thì cho ở mức 3) để xác

định vai trò của học luôn (ví dụ "CBCT" thì cho L_i là 1).

- **Đối Campus_like** : thì ta cho sinh ngẫu nhiên nhận 1 trong 3 giá trị 1-Không thích , 2-Bình thường, 3-Thích.
- **Đối với Cap_j_r** thì lấy từ việc tính các vai trò có trong ca thi j

Ngoài ra, phần tiền xử lý dữ liệu này con đo độ hợp lý của đầu vào qua phần **Tiền kiểm tra tính khả thi tĩnh (Static Audit)**: Trước khi trả dữ liệu về bộ điều phối, thuật toán đếm tổng số cán bộ rảnh rỗi ($B_{ij} \neq 2$) và so sánh với định mức nhu cầu nhân sự. Nếu thiếu hụt nguồn cung, hệ thống sẽ cảnh báo lỗi dữ liệu cục bộ thay vì để bộ giải chạy mù quáng.

3.3 Cấu trúc chương trình và Bộ giải

Hệ thống được thiết kế theo mô hình phân lớp module (*Modular Design Pattern*), tách biệt rõ ràng giữa khâu tiền xử lý dữ liệu, xây dựng mô hình toán học lý thuyết và quá trình thực thi tối ưu hóa. Ngôn ngữ lập trình chính được sử dụng để hiện thực hóa hệ thống là Python, kết hợp với thư viện mô hình hóa toán học tuyến tính và bộ giải mã nguồn mở hiệu năng cao CBC (*Coin-or branch and cut*).

Toàn bộ luồng thực thi của ứng dụng được cấu trúc đồng bộ thông qua các tệp tin cốt lõi. Kiến trúc hệ thống và mã giả (*Pseudo-code*) mô tả thuật toán logic của từng phân hệ được quy định cụ thể như sau:

3.3.1 Mã giả toàn diện hệ thống (System Orchestration)

Dưới đây là luồng phối hợp hoạt động của hệ thống, được điều khiển bởi module main.py. Quá trình bao gồm việc tương tác với người dùng để điều chỉnh các tham số trọng số và tự động đánh giá chất lượng nghiệm.

```
1 THUAT TOAN DieuPhoiHeThongTrungTam():
2     Input: Tệp Excel du lieu "Dataset_Anonymized_Invigilator_
3           Assignment_Problem.xlsx"
4     Output: Trang thai hoan thanh (True/False)
5     Begin
6         // Buoc 1: Doc va Tien xu ly du lieu
7         DataModel <- TienXuLyDuLieu(Input)
8
9         // Buoc 2: Menu dieu chinh du lieu thu cong
10        MenuDieuChinhDuLieu(DataModel)
11
12        // Buoc 3: Kiem tra tien kha thi (Static Audit)
```

```
12      IsFeasible, AuditErrors <- KiemTraTienKhaThi(DataModel)
13
14      IF NOT IsFeasible THEN:
15          PRINT "DUNG HE THONG: Phat hien loi du lieu tinh (
              Infeasible)."
16          RETURN False
17      END IF
18
19      // Buoc 4: Vong lap Giai mo hinh va Tinh chinh trong so
20      KhoiTao TrongSo(theta, TAX_LACK, TAX_BUSY, TAX_QUAL) // Gia
              tri mac dinh
21      LichSuTuning <- [] // Luu vet de goi y cai thien
22
23      WHILE True DO:
24          // Xay dung va tien hanh giai mo hinh
25          Prob, X <- XayDungMoHinhILP(DataModel, TrongSo)
26          Status, Metrics <- GiaoTiepBoGiai(Prob, X, DataModel)
27
28          // In bao cao hieu nang
29          PRINT "TRANG THAI: " + Status
30          PRINT "Do lech (Gap): " + Metrics.Gap
31          PRINT "Cac vi pham rang buoc mem (Slacks): " + Metrics.
              Slacks
32          HienThiGoiYTinhChinh(Metrics, LichSuTuning)
33
34          // Xu ly lua chon cua nguoi dung
35          LuaChon <- NHAP_LUA_CHON(1: Xuat Excel, 2: Dieu chinh
              trong so, 0: Thoat)
36
37          IF LuaChon == 1 THEN:
38              GhiRaExcel(Prob, X, DataModel)
39              BREAK
40          ELSE IF LuaChon == 2 THEN:
41              TrongSo <- CapNhatTrongSoTuMenu(TrongSo)
42              LuuVaoLichSu(LichSuTuning, TrongSo, Metrics)
43          ELSE:
44              BREAK // Nguoi dung chon Thoat (0)
45          END IF
46      END WHILE
47
48      RETURN True
49  End
```

Listing 1: Mã giả luồng thực thi Điều phối hệ thống (Orchestrator) tại tệp main.py

3.3.2 data_preprocessing.py (Tiền xử lý dữ liệu)

Phân hệ này đảm nhiệm toàn bộ các tác vụ làm sạch dữ liệu thô, thực hiện thuật toán nội suy các trường thông tin bị khuyết thiếu, cô lập các tập hợp rời rạc và số hóa các ma trận tham số đầu vào.

```
1 THUAT TOAN TienXuLyDuLieu(excel_file):
2     Input: Tep tin du lieu thoi "Dataset.xlsx"
3     Output: Cau truc du lieu DataModel
4     Begin
5         df <- Doc_DataFrame_Tu_Excel(excel_file)
6
7         // 1. Chuan hoa du lieu va Khoi tao cac tap hop (Sets)
8         Chuan_Hoa_Du_Lieu_Thoi_Gian(df)
9         CB <- Trich_Xuat_Can_Bo(df)
10        K  <- Trich_Xuat_Co_So(df)
11        CT <- Tao_ID_Ca_Thi_Duy_Nhat(df) // ID = MaCaThi + CoSo
12        R  <- Trich_Xuat_Va_Chuan_Hoa_Vai_Tro(df)
13
14        // 2. Trich xuất tham số thực tế (Parameters)
15        Cap_jr <- Tinh_Nhu_Cau_Nhan_Su_Tung_Ca_Va_Vai_Tro(df)
16        CT_info <- Lay_Thong_Tin_Chi_Tiet_Ca_Thi(df)
17
18        // 3. Phat sinh du lieu gia lap (Synthetic Data)
19        B_ij <- Phat_Sinh_Trang_Thai_Ban(CB, CT, soft_rate, hard_rate)
20        L_i  <- Xac_Dinh_Nang_Luc_Chuyen_Mon_Tu_Lich_Su(CB, df)
21        Campus_like_ik <- Phat_Sinh_So_Thich_Co_So(CB, K)
22
23        // Dong goi du lieu
24        DataModel <- {
25            Sets: [CB, CT, R, K],
26            Parameters: [Cap_jr, B_ij, CT_info],
27            Synthetic: [L_i, Campus_like_ik]
28        }
29        RETURN DataModel
30    End
31
32 THUAT TOAN KiemTraTienKhaThi(DataModel):
33     Input: Cau truc du lieu DataModel
34     Output: IsFeasible, DanhSachLoi
35     Begin
36         Errors <- []
37         CB, CT, R, Cap_jr, B_ij, L_i <- Phan_Ra(DataModel)
38
```

```

39     FOR EACH ca_thi j IN CT DO:
40         // Kiem tra 1: Dieu kien can ve so luong (Pigeonhole)
41         NhuCauTong <- Tong_Nhan_Su_Yeu_Cau(j, Cap_jr)
42         CungUngTong <- Dem_So_Can_Bo(i IN CB sao cho B_ij[i][j] !=
           2) // Khong ban tuyet doi
43
44         IF NhuCauTong > CungUngTong THEN:
45             Errors.Append("Ca " + j + ": Thieu hut nhan su tong
              the.")
46             CONTINUE // Bo qua kiem tra chuyen mon vi da thieu so
              luong
47         END IF
48
49         // Kiem tra 2: Dieu kien du ve chat luong (Lead Proctor)
50         FOR EACH vai_tro r IN R yeu cau boi ca j DO:
51             NangLucYeuCau <- Lay_Trinh_Do_Tu_Tu_Dien(r)
52             SoNguoiDapUng <- Dem_So_Can_Bo(i IN CB sao cho B_ij[i
              ][j] != 2 AND L_i[i] >= NangLucYeuCau)
53
54             IF SoNguoiDapUng == 0 THEN:
55                 Errors.Append("Ca " + j + ": Khong co can bo ranh
              viec du trinh do cho vai tro " + r)
56             END IF
57         END FOR
58     END FOR
59
60     IsFeasible <- (Errors.Length == 0)
61     RETURN IsFeasible, Errors
62 End

```

Listing 2: Mã giả quy trình Tiền xử lý và Kiểm tra tính tại tệp data_preprocessing.py

3.3.3 model_builder.py (Xây dựng mô hình toán học)

Đây là module cốt lõi của hệ thống, đóng vai trò "dịch thuật" toàn bộ các khái niệm tập hợp, ma trận tham số logic thành hàng chục nghìn biến quyết định nhị phân x_{ijr} , đồng thời nhúng hệ thống phương trình ràng buộc cùng hàm mục tiêu tổng hợp vào bộ nhớ solver.

```

1  THUAT TOAN XayDungMoHinhToanHoc(Data, Weights):
2      Input: Cau truc du lieu Data, Cau hình trong so Weights
3      Output: Bai toan prob, Khong gian bien X, Bien Slacks
4      Begin
5          prob <- Khoi_Tao_Bai_Toan_Toi_Thieu_Hoa(Objective = Minimize)

```

```
6
7 // Khoi tao khong gian bien quyet dinh (Ap dung Lazy Init)
8 X <- Khoi_Tao_Mang_Bien_Nhi_Phan(Data.Sets.CB, Data.Sets.CT,
  Data.Sets.R)
9 Slacks <- Khoi_Tao_Bien_Slack_Noi_Long_Nguyen_Duong()
10
11 // --- THIET LAP RANG BUOC CHAT (HARD CONSTRAINTS) ---
12 FOR EACH ca_thi j, vai_tro r DO:
13   Gan_Rang_Buoc_Nhu_Cau: Tong_X_ijr + slack_cap == Cap_jr
14 END FOR
15
16 Kich_Hoat_Lead_Proctor_Rule(X) // Ep buoc >= 1
  can bo Qualified
17 Khoa_Bien_Theo_Ma_Tran_Ban(X, Data.BanMatrix) // X_ijr = 0 neu
  B_ij = 2
18 Cam_Chong_Lan_Thoi_Gian_Va_Di_Chuyen_Bat_Kha_Thi(X, gap < 2h)
19
20 // --- THIET LAP RANG BUOC MEM VA HAM MUC TIEU (SOFT
  CONSTRAINTS) ---
21 X_sum <- Khoi_Tao_Hash_Map_Tinh_Toan_Nhanh(X) // Tinh toan W_i
  , W_high, W_low
22
23 Pair_Penalties <- Loc_Ma_Tran_Phats_Cap_Di_Chuyen(Chi tao Y_
  pair neu Penalty > 5.0)
24 Thiet_Lap_Luat_Bac_Cau_Met_Moi(Y_fatigue_3, Y_fatigue_4, Y_
  fatigue_5)
25
26 // Cau truc Ham muc tieu tong hop
27 Objective <- Weights.omega * F2 + Weights.TAX * Slacks +
  Weights.theta * (W_high - W_low)
28 prob.Gan_Ham_Muc_Tieu(Objective)
29
30 RETURN prob, X, Slacks
31 End
```

Listing 3: Mã giả xây dựng hệ phương trình tối ưu tại tệp model_builder.py

3.3.4 solver.py (Giao tiếp bộ giải và Xuất kết quả)

Khác với các cấu trúc triển khai thông thường chỉ ghi nhận kết quả thô từ bộ giải ngoại vi, module này chịu trách nhiệm quản lý vòng đời tính toán, cấu hình điều kiện ngắt thuật toán phân nhánh và cắt (*Branch-and-Cut*) để chống treo máy. Đồng thời, module thực hiện phân loại nghiệm một cách học thuật dựa trên tính toán chỉ số tiệm

cận tối ưu (*Relative MIP Gap*) và trích xuất định lượng các vi phạm ràng buộc mềm để chuyển giao ngược lại cho bộ điều phối trung tâm.

```
1 THUAT TOAN GiaoTiepBoGiai(prob, X, TimeLimit=50s):
2     Input: Bai toan toi uu prob, Ma tran bien X, Thoi gian gioi han
           TimeLimit
3     Output: Cau truc ket qua Result (Status, ActualGap, Metrics)
4     Begin
5         // Kích hoạt trình giải CBC với sai số mục tiêu mặc định 2%
6         Thuc_Thi_Solver_CBC(prob, TimeLimit=60s, GapRel=0.02)
7
8         // Tính toán do lệch tối ưu thực tế (Actual MIP Gap)
9         ActualGap <- Abs(prob.ObjValue - prob.BestBound) / Abs(prob.
           ObjValue)
10
11        // Phân loại chất lượng nghiệm học thuật
12        IF ActualGap < 0.001 THEN:
13            Status <- "Optimal"           // Nghiệm tối ưu toàn cục
14        ELSE IF ActualGap <= 0.02 THEN:
15            Status <- "Near-Optimal"     // Đạt chuẩn hồi tu công
           nghiệp
16        ELSE:
17            Status <- "Feasible"         // Chạm TimeLimit, chưa hồi tu
           sau
18        END IF
19
20        // Trích xuất và thống kê định lượng các vi phạm hành chính
21        So_ca_vi_pham <- Dem_So_Bien_Slack_Lon_Hon_Zero(prob)
22
23        Result <- {
24            Status: Status,
25            ActualGap: ActualGap,
26            SlackViolations: So_ca_vi_pham
27        }
28
29        RETURN Result
30    End
```

Listing 4: Mã giả thực thi bộ giải và phân loại nghiệm khoa học tại tệp solver.py

3.3.5 Các kỹ thuật lập trình thực tiễn

Để chuyển hóa mô hình toán học lý tưởng thành mã nguồn có khả năng vận hành ổn định trên dữ liệu thực tế mang tính NP-Hard, nhóm đã tích hợp trực tiếp hai kỹ thuật

lập trình cấp cao vào mã nguồn:

- **Kỹ thuật Nới lỏng Ràng buộc (Constraint Relaxation & Big-M):** Trong file `model_builder.py`, các ràng buộc Cứng về nhu cầu nhân sự, trạng thái bận nhẹ và năng lực đã được chuyển hóa thành ràng buộc Mềm bằng cách thêm các biến nới lỏng (Slack Variables). Cụ thể: `slack_cap` cho phép thiếu người, `slack_busy` cho phép ép người bận nhẹ đi làm, và `slack_qual` chấp nhận vượt cấp chuyên môn. Hàm tối ưu điểm phạt F_2 sẽ được cộng dồn hệ thống trọng số phạt cực lớn (10.000 điểm cho thiếu người, 5.000 điểm cho ép đi làm). Cơ chế nới lỏng này mang lại ưu điểm vượt trội: triệt tiêu đi phần nào sự cố sập đổ hệ thống (*Infeasible*) và cung cấp phương án "Tiệm cận tốt nhất" (Best-effort Heuristic) giúp người dùng dễ dàng nhận diện "điểm nghẽn" để can thiệp thủ công.
- **Kỹ thuật Tối ưu hóa hiệu năng (Pre-calculated Sums & RAM Optimization):** Quá trình biên dịch mô hình đòi hỏi hệ thống quét qua không gian biến quyết định hàng chục nghìn lần. Để tránh hiện tượng "Computation Bottleneck", nhóm áp dụng kỹ thuật Caching. Hệ thống sử dụng Hash Map để lưu trữ biến tổng `X_sum[i, j]`, giúp các vòng lặp phía sau có thể truy xuất trực tiếp với độ phức tạp $O(1)$. Đồng thời, Lazy Initialization được dùng để chỉ khởi tạo biến `slack_busy` tại những tọa độ thực sự có lịch bận. Kỹ thuật này đã thành công trong việc nén không gian tìm kiếm, giảm thời gian xây dựng mô hình từ hơn 5 phút xuống còn dưới 3 giây.

4 Phân tích hiệu năng

Phân tích hiệu năng là một bước quan trọng trong quy trình tối ưu hóa. Quá trình đánh giá hiệu năng cung cấp một bức tranh định lượng trực quan, phản ánh sự cải thiện của mô hình đề xuất so với hệ thống cũ. Trong thực tế, đây là cơ sở dữ liệu để thuyết phục khách hàng cũng như các bên liên quan về tính khả thi, độ tin cậy và giá trị kinh tế mà mô hình mang lại.

Trong phần này, nhóm sẽ đánh giá toàn diện về hiệu năng của mô hình bằng cách phân tích độ phức tạp, đo lường thời gian và so sánh với baseline.

4.1 Phân tích độ phức tạp

Độ phức tạp của thuật toán mô tả mức độ tiêu tốn tài nguyên (thời gian, bộ nhớ). Công cụ để phân tích độ phức tạp là Big-O notation. Big-O giúp xác định xu hướng tăng trưởng về chi phí của một thuật toán khi kích thước dữ liệu đầu vào thay đổi, tập trung

vào thành phần tăng trưởng nhanh nhất.

Việc phân tích độ phức tạp giúp nhóm có thể đo lường sự tăng trưởng của tài nguyên, từ đó cung cấp cơ sở để so sánh các giải pháp khác nhau cho cùng một bài toán và chọn ra phương án hiệu quả nhất. Bên cạnh đó, việc phân tích còn giúp xác định được giới hạn chịu tải tối đa của thuật toán để chuẩn bị phương án dự phòng tốt nhất. Nếu không phân tích trước, khi quy mô tăng lên (ví dụ như số lượng cán bộ, ca thi tăng) thì hệ thống có thể bị sập hoặc quá tải.

Ký hiệu sử dụng

Ký hiệu	Ý nghĩa	Giá trị thực tế
N	Số dòng dữ liệu Excel	769
$ CB $	Số cán bộ	73
$ CT $	Số ca thi	83
$ R $	Số vai trò	3
V	Số biến quyết định = $ CB \times CT \times R $	18.177
C	Số ràng buộc	~ hàng chục nghìn

Bảng 4.13 Ký hiệu và giá trị thực tế của bài toán

4.1.1 Phân tích độ phức tạp thời gian

Độ phức tạp thời gian của hệ thống được phân tích qua 3 giai đoạn thực thi chính: tiền xử lý, xây dựng mô hình và giải bài toán.

Giai đoạn 1 — Tiền xử lý dữ liệu (`data_preprocessing.py`)

Bước	Thao tác chính	Độ phức tạp	Ghi chú
Đọc & chuẩn hóa	<code>read_excel</code> , <code>apply(normalize)</code>	$O(N)$	$N = 769$ dòng dữ liệu
Trích xuất CB, CT, ...	<code>unique()</code> , <code>groupby()</code> , <code>drop_duplicates()</code>	$O(N)$	
Xây dựng B_{ij}	Định nghĩa từ điển + <code>random.sample</code>	$O(CB \times CT)$	$73 \times 83 = 6.059$ phần tử
Xây dựng L_i	Lọc df theo từng CB	$O(CB \times N)$	Bước chậm nhất GD1: $73 \times 769 \approx 56.137$
Xây dựng <code>Campus_like_ignore</code>	<code>random.choice + 2</code> gán/CB	$O(CB)$	
Tổng GD1		$O(CB \times N)$	Dominant term: L_i

Bảng 4.15 Độ phức tạp Giai đoạn 1 – Tiền xử lý dữ liệu

Giai đoạn 2 — Xây dựng mô hình (`model_builder.py`)

Bước	Thao tác chính	Độ phức tạp	Ghi chú
Khởi tạo biến X , Slacks	<code>LpVariable.dicts</code>	$O(V)$	$V = 18.177$ biến
RC1 – Nhu cầu	<code>IpSum</code> theo CB cho mỗi (j, r)	$O(V)$	
RC2 – Chống trùng lịch	Tính trước <code>overlapping_pairs</code> (vòng lặp đôi trên CT), sau đó duyệt CB	$O(CT ^2 + CB \times P_o)$	$ P_o $ = số cặp ca chồng lẫn thực tế
RC3 – Bận việc	Chỉ kích hoạt khi $B_{ij} = 1$ (5%)	$O(0,05 \times V)$	Worst case $O(V)$
RC4 – Năng lực	Điều kiện theo level CB	$O(CB \times CT)$	
RC5 – Di chuyển bất khả thi	Vòng lặp đôi trên CT , thêm ràng buộc cho từng CB	$O(CT ^2 \times CB)$	$83^2 \times 73 \approx 502.897$ – ràng buộc đắt nhất GD2
Ma trận phạt P_{ijr}	Vòng lặp 3 lớp trên $CB \times CT \times R$	$O(V)$	
Tính <code>pair_penalties</code>	Vòng lặp đôi trên CT (không có triplet)	$O(CT ^2)$	$83^2 = 6.889$ – chỉ duyệt cặp ca, không có vòng lặp 3 lớp
Biến cờ Y_{pair} + ràng buộc kích hoạt	$ CB \times P $ ràng buộc, $ P $ = số cặp có phạt	$O(CB \times P)$	$ P \leq CT ^2$, thường $ P \ll CT ^2$
Tổng GD2		$O(CT ^2 \times CB)$	Dominant: RC5 (≈ 502.897)

Bảng 4.17 Độ phức tạp Giai đoạn 2 – Xây dựng mô hình

Giai đoạn 3 — Giải bài toán (`solver.py`)

Độ phức tạp lý thuyết worst case: $O(2^V)$ – duyệt toàn bộ cây nhị phân với V lá.

Thực tế với CBC: chỉ liệt kê **911 nodes** (trong tổng số $2^{18.177}$ nút lý thuyết) nhờ các kỹ thuật cắt nhánh.

Tổng Giai đoạn 3 (lý thuyết): $O(2^V)$.

Độ phức tạp thời gian tổng thể (lý thuyết): $O(2^V) = O(2^{|CB| \times |CT| \times |R|})$

4.1.2 Phân tích độ phức tạp không gian

Độ phức tạp không gian được xác định bởi dung lượng bộ nhớ cần thiết để lưu trữ các ma trận và cấu trúc dữ liệu của mô hình ILP.

Thành phần chiếm ưu thế	Độ phức tạp	Thực tế
Dữ liệu đầu vào	$O(N + CB \times CT)$	~ 6.828 entries
Biến quyết định ($X +$ Slacks)	$O(V) = O(CB \times CT \times R)$	$3 \times 18.177 = 54.531$ biến
Ma trận phạt P_{ijr}	$O(V)$	18.177 entries
Biến cờ Y_{pair}	$O(CB \times CT ^2)$	$73 \times P , P \leq 83^2 = 6.889$
Ma trận ràng buộc CBC (sparse)	$O(C \times V)$ worst case	~400–800 MB RAM
Độ phức tạp không gian tổng thể: $O(C \times V) = O(C \times CB \times CT \times R)$, trong đó C là số ràng buộc (bị chi phối bởi RC5: $O(CT ^2 \times CB)$)		

Bảng 4.19 Độ phức tạp không gian

4.1.3 Nhận xét

Với quy mô thực tế của bài toán (73 cán bộ, 83 ca thi, 3 vai trò, 18.177 biến nhị phân), solver CBC vẫn tìm được lời giải tối ưu toàn cục trong 61,04 giây với giá trị hàm mục tiêu $Z^* = 2.678,33$ – hoàn toàn phù hợp cho ứng dụng thực tế.

Tuy nhiên, do bài toán thuộc lớp **NP-Hard**, khi số lượng cán bộ hoặc ca thi tăng lên gấp đôi, thời gian giải tăng theo cấp số nhân $O(2^{|CB| \times |CT| \times |R|})$. Lúc đó mô hình sẽ cần được tối ưu bằng cách chia nhỏ bài toán hoặc điều chỉnh lại các ràng buộc.

Sự đánh đổi giữa thời gian và không gian: Khi tạo thêm nhiều ràng buộc và biến phụ, độ phức tạp không gian tăng lên ($O(C \times V)$), nhưng mô hình mô tả bài toán chặt hơn, hỗ trợ solver tìm ra nghiệm nhanh hơn, từ đó giảm độ phức tạp thời gian thực tế.

4.2 Đánh giá thời gian thực thi

Để kiểm định solver có thể giải dataset trong thời gian hợp lý và dự báo thời gian khi mô hình mở rộng, nhóm thực hiện đo lường thời gian CBC Solver giải mô hình ILP với các quy mô khác nhau.

Phương pháp đo

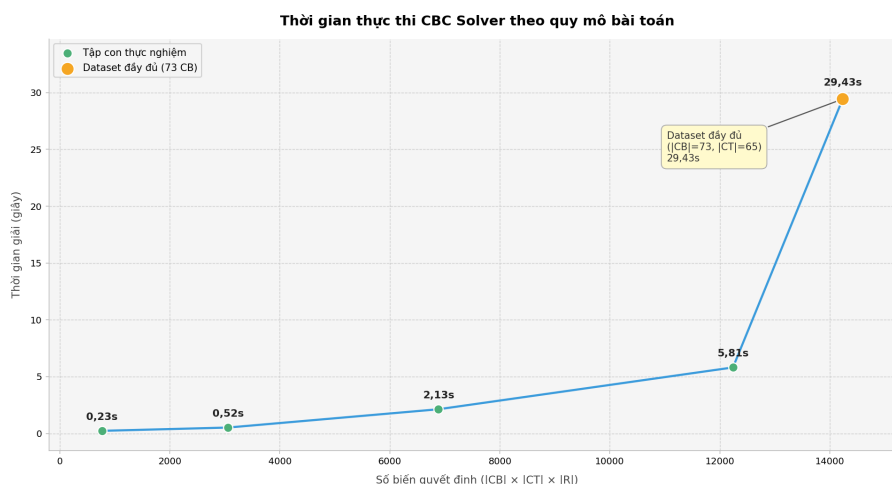
Nhóm thí nghiệm trên tập con dữ liệu với quy mô tăng dần bằng cách giới hạn số cán bộ và ca thi theo tỷ lệ cố định $|CB| : |CT| \approx 0,88$ (73 : 83). Ở mỗi quy mô, sau

khi cắt danh sách CB và CT, nhóm lọc đồng bộ các tham số phụ thuộc gồm B_{ij} , Cap_jr, CT_info, L_i , Campus_like_ik để chỉ giữ lại các phần tử thuộc tập con tương ứng. Thời gian được ghi nhận từ dòng ở trong output dòng Time (Wallclock seconds) sau khi chạy file evaluate_time.py

Kết quả đo thời gian

$ CB $	$ CT $ subset	Thời gian (s)	Trạng thái
15	17	0.13	Optimal
30	34	1.23	Optimal
45	51	5.62	Optimal
60	68	7.15	Optimal
73	65	29,51	Optimal

Bảng 4.21 Thời gian thực thi CBC theo kích thước subset (hàng xám = dataset đầy đủ)



Hình 4.1 Biểu đồ thời gian thực thi CBC Solver theo kích thước bài toán

Nhận xét: Với dataset đầy đủ gồm 73 CB, 83 ca thi, 18.177 biến nhị phân ($= 73 \times 83 \times 3$), CBC solver đạt trạng thái Optimal trong 29.51 giây – hoàn toàn phù hợp cho ứng dụng thực tế. Xu hướng tăng trưởng thời gian mang tính gần tuyến tính ở quy mô nhỏ (dưới 60 CB) và bắt đầu tăng nhanh hơn khi vượt 60 CB, phản ánh bản chất NP-Hard của bài toán. Trong Phần 10 nhóm sẽ thảo luận về các phương án mở rộng khi quy mô tăng đáng kể.

4.3 So sánh với Baseline

Tiêu chí so sánh

Ký hiệu	Tên tiêu chí	Cách tính trong code	Ý nghĩa thực tế
M1	Tổng điểm phạt	$M1 = \text{penalty_static} + \text{penalty_pairs} + \text{penalty_fatigue}$	Tổng hợp toàn bộ các loại điểm phạt động và phạt tĩnh trong hệ thống – nhỏ hơn là tốt hơn
M2	Độ lệch khối lượng công việc	$W_{\max} - W_{\min}$	Đo lường tính công bằng trong phân phối công việc – nhỏ hơn là tốt hơn
M3	Penalty tĩnh	Tích lũy $\text{pen_campus} + \text{pen_qual}$ mỗi dòng phân công	Tổng mức không phù hợp về sở thích cơ sở và lãng phí năng lực – nhỏ hơn là tốt hơn
M4	Penalty mệt mỏi	Phạt lũy tiến theo cấu trúc hình bậc thang	Tổng điểm phạt khi CB trực nhiều hơn 2 ca một ngày – nhỏ hơn là tốt hơn
M5	Penalty di chuyển và lãng phí thời gian chờ	Phạt 50 điểm khi vừa đủ thời gian di chuyển (2.0 giờ), giảm dần 10 điểm/giờ. Phạt 15 điểm/giờ ngồi chờ	Tổng điểm phạt khi CB phải di chuyển giữa 2 ca thi trong một ngày hoặc phải chờ hơn 2 tiếng giữa 2 ca – nhỏ hơn là tốt hơn
M6	Vi phạm nhu cầu nhân sự	Với mỗi cặp (j, r) , so sánh số người thực tế với Cap_jr ; nếu thiếu $\rightarrow$ số chỗ thiếu $\times 10.000$	Số chỗ thiếu nhân sự trong lịch – nhỏ hơn là tốt hơn
M7	Vi phạm trạng thái bận	Duyệt từng dòng; nếu $B_{ij}[(i, j)] = 1 \rightarrow$ cộng 5.000	Số lần ép CB đang bận đi làm – nhỏ hơn là tốt hơn
M8	Vi phạm năng lực	Duyệt từng dòng; nếu $L_i < \text{req_level_r} \rightarrow$ cộng 5.000	Số lần phân công không đủ năng lực – nhỏ hơn là tốt hơn

Bảng 4.23 Tám tiêu chí so sánh Baseline vs. PuLP Solver

Phương pháp so sánh

Lấy dữ liệu từ lịch giải bằng Solver:

Sau khi Solver giải xong sẽ xuất ra các tệp tin bao gồm:

- Tệp `solver_metrics.json`: Lưu trữ các thông tin về số lần vi phạm và điểm phạt.
- Tệp `B_ij_cache.json`: Lưu trữ dữ liệu cán bộ bận vào ca nào (được tạo bằng phương pháp giả lập sinh số ngẫu nhiên).
- Tệp `Campus_like_cache.json`: Lưu trữ dữ liệu sở thích về địa điểm coi thi của cán bộ (được tạo bằng phương pháp giả lập sinh số ngẫu nhiên).

Việc sử dụng chung một bộ dữ liệu đầu vào này sẽ giúp cho việc đối sánh giữa hai phương án lịch được nhất quán.

Lấy dữ liệu từ lịch Baseline:

Hệ thống sẽ quét lịch gốc và chạy các công thức tính điểm tiêu chí, sau đó lưu vào cấu trúc dữ liệu dict tên `baseline_metrics`.

Thông số sử dụng chạy chương trình và so sánh:

Trọng số hàm mục tiêu:

ω : 1,0 - Trọng số thành phần phạt chất lượng F_2

θ : 20,0 - Trọng số thành phần công bằng F_1

Hệ số phạt (Penalty Tax):

`TAX_LACK_STAFF`: 10.000 - Thiếu người gác ca (`slack_cap > 0`)

`TAX_FORCE_BUSY`: 5.000 - Ép cán bộ Soft-Busy đi làm ($B_{ij} = 1$)

`TAX_BAD_QUAL`: 5.000 - Phân công sai chuyên môn (`slack_qual`)

Tỷ lệ B_{ij} (trạng thái bận) dùng trong phân tích chính:

`soft_busy_rate`: 5%, $B_{ij} = 1$ – bận nhẹ (có thể ép)

`hard_busy_rate`: 2%, $B_{ij} = 2$ – bận tuyệt đối

Tỷ lệ `Campus_like_ik` (sở thích cơ sở):

Giá trị sở thích được sinh ngẫu nhiên đều theo ba nhóm với xác suất 13 mỗi nhóm:

`Nhom_CS1`: 3 (thích) - 1 (ghét) ($\approx 33\%$ cán bộ).

`Nhom_CS2`: 1 (ghét) - 3 (thích) ($\approx 33\%$ cán bộ).

Nhom_CanBang: 2 (bình thường) - 2 (bình thường) ($\approx 33\%$ cán bộ).

Mức năng lực L_i và yêu cầu theo vai trò:

CBCT: 1

Thuky: 2

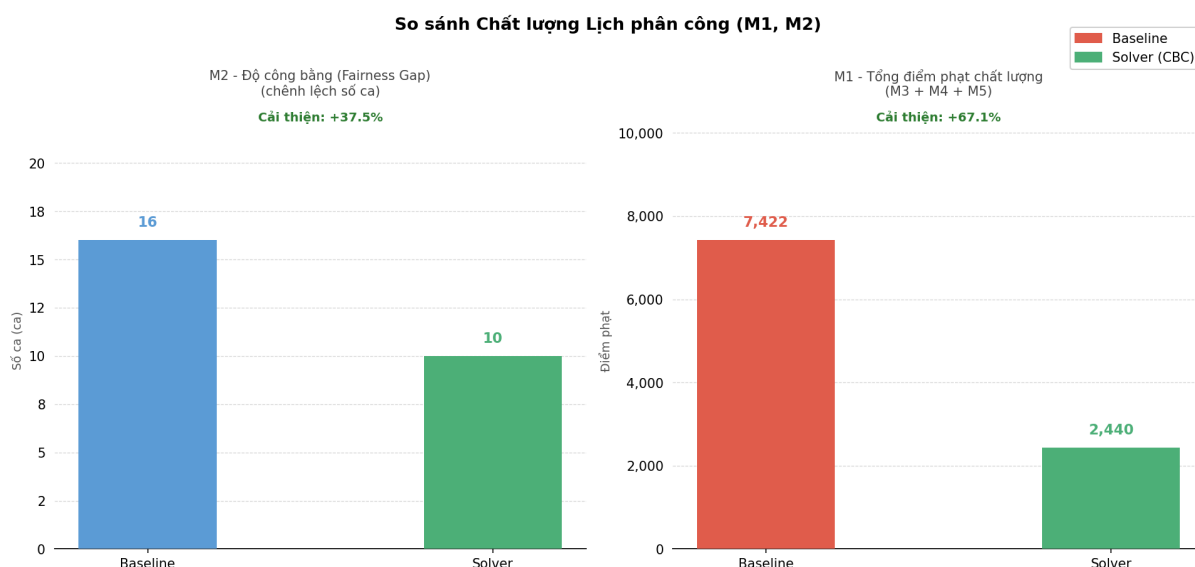
TruongHD: 3

Kết quả so sánh

KH	Tiêu chí	Baseline	Solver	Cải thiện	Hướng
M1	Tổng điểm phạt	7,422	2,440	-67,1%	↑ tốt hơn
M2	Độ lệch khối lượng công việc	16	10	-37,5%	↑ tốt hơn
M3	Penalty tĩnh	3,717	1,675	-54,9%	↑ tốt hơn
M4	Penalty một mỗi	3120.0	0,00	-100,0%	↑ tốt hơn
M5	Penalty di chuyển và lãng phí thời gian chờ	585.0	765.0	+30,77%	↓ xấu hơn
M6	Vi phạm nhu cầu nhân sự	0,00	0,00	—	—
M7	Vi phạm trạng thái bận	52	0	-100%	↑ tốt hơn
M8	Vi phạm năng lực	0,00	0,00	—	—

Bảng 4.25 Kết quả so sánh định lượng Baseline vs. PuLP Solver (CBC, Optimal)

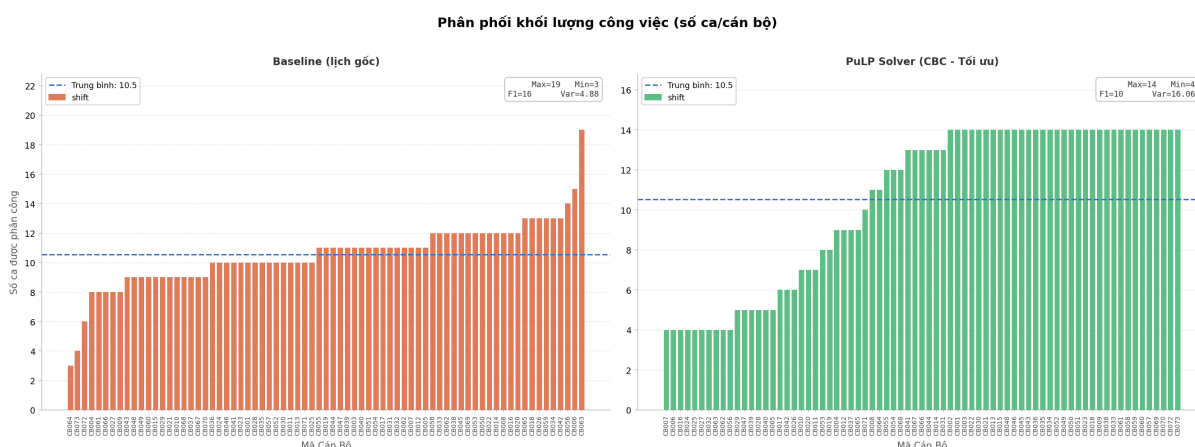
Phân tích và nhận xét



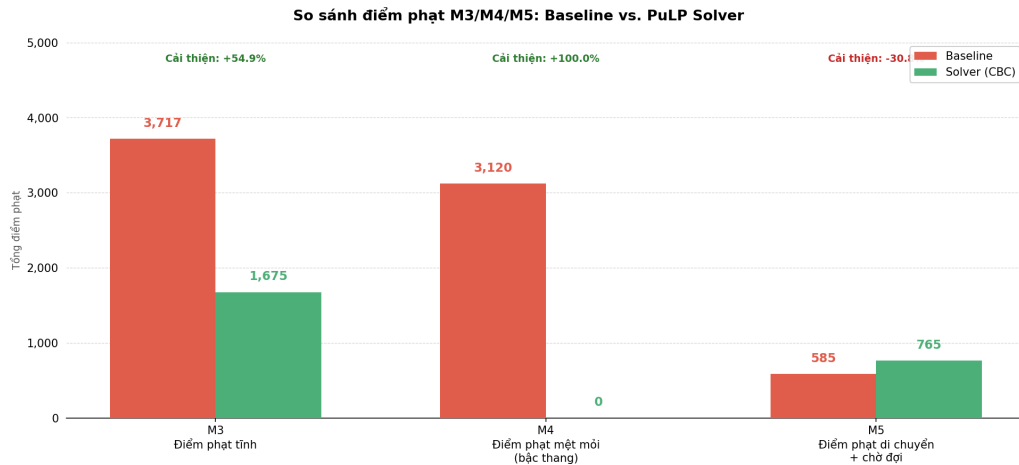
Hình 4.2 So sánh chất lượng lịch phân công – Baseline vs. PuLP Solver

M1 – Tổng điểm phạt cải thiện **67,1%**, Tổng các loại điểm phạt động và điểm phạt tĩnh giảm đáng kể. Lịch mới có chất lượng phân công và mức độ hài lòng của cán bộ coi thi (CBCT) cao hơn.

M2 – Độ lệch khối lượng công việc cải thiện **37,5%** – từ chênh lệch 16 ca xuống còn 10 ca. Tuy nhiên, khi nhìn vào biểu đồ phân phối khối lượng công việc dưới đây, có thể thấy phương sai khối lượng công việc cao làm một nhóm cán bộ phải coi thi quá nhiều ca trong khi nhóm khác coi thi quá ít ca.



Hình 4.3 Phân phối khối lượng công việc – Baseline vs. PuLP Solver

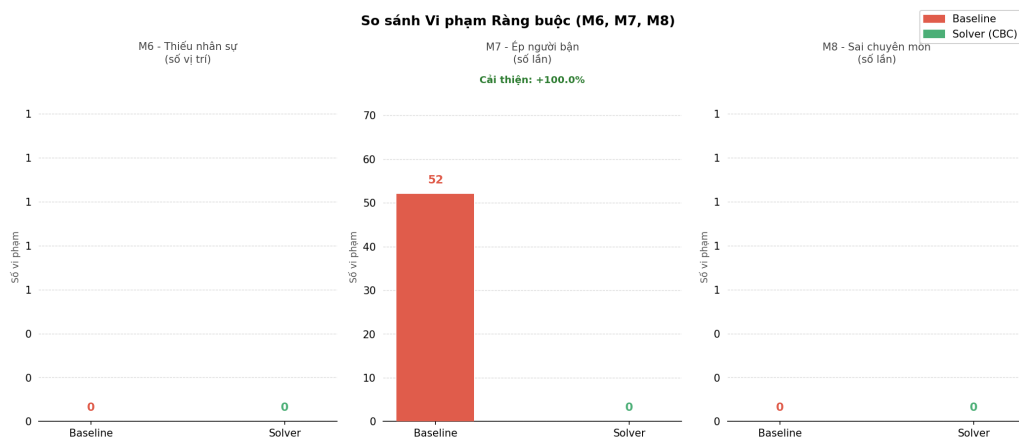


Hình 4.4 So sánh điểm phạt: Baseline vs. PuLP Solver (CBC, Optimal)

M3 – Penalty tình giảm **54,9%**, từ 3717 xuống 1675 điểm. Điều này đồng nghĩa với việc hơn một nửa số lần phân công sai chỗ hoặc sai người trong lịch gốc đã được solver loại bỏ, cán bộ được coi thi gần nhà hoặc tại cơ sở quen thuộc hơn, giảm bất tiện đi lại và tăng sự hài lòng, người có kinh nghiệm cao được giao vai trò xứng đáng hơn, tránh lãng phí nhân lực và tăng hiệu quả giám sát phòng thi.

M4 – Penalty mệt mỏi giảm **100%** – không còn trường hợp cán bộ phải làm trên 2 ca liên tiếp tại cùng một cơ sở trong một ngày.

M5 – Penalty di chuyển và lãng phí thời gian chờ tăng **30.77%**, từ 585 lên 765 điểm. Số lần cán bộ phải di chuyển giữa 2 cơ sở trong ngày hay cán bộ phải chờ giữa 2 ca tăng đáng kể.



Hình 4.5 Phân tích vi phạm ràng buộc cứng: Baseline vs. Solver

M6 – Vi phạm nhu cầu nhân sự: Cả 2 lịch phân công đều đảm bảo không có

ca nào bị thiếu cán bộ so với yêu cầu tối thiểu.

M7 – Vi phạm trạng thái bận: Lịch gốc có 52 trường hợp ép người bận, còn lịch tối ưu giảm xuống không còn trường hợp nào.

M8 – Vi phạm năng lực: Cả 2 lịch đều đảm bảo cán bộ được phân công có năng lực phù hợp với yêu cầu của ca.

Kết luận: PuLP Solver có khả năng vượt trội trên **5/8 tiêu chí** đánh giá, với cải thiện đặc biệt ấn tượng ở M4 (100%) và M7 (100%), đồng thời duy trì tính khả thi vận hành (M6, M8: Optimal). Hướng phát triển tiếp theo cần tập trung cân bằng tối ưu hóa chất lượng với phân phối workload công bằng, và giải quyết triệt để mâu thuẫn dữ liệu đầu vào.

4.4 Tỷ lệ Feasible/Infeasible theo Busy Rate

Mục tiêu thí nghiệm

Trong mô hình ILP đã xây dựng, tham số $B_{ij} \in \{0, 1, 2\}$ mô tả trạng thái bận của cán bộ i tại ca thi j theo hai mức: $B_{ij} = 1$ (bận nhẹ – Soft-Busy) cho phép phân công kèm chi phí phạt $TAX_FORCE_BUSY = 5.000$, còn $B_{ij} = 2$ (bận tuyệt đối – Hard-Busy) là ràng buộc cứng tuyệt đối cấm phân công.

Khi tỷ lệ Hard-Busy tăng cao, không gian nghiệm khả thi của bài toán bị thu hẹp dần – đến một ngưỡng nào đó, ràng buộc nhân sự tối thiểu (LeadProctor) không thể được thỏa mãn dù đã có slack, dẫn đến mô hình trở thành **Infeasible**. Xác định ngưỡng tối hạn này có ý nghĩa khoa học và thực tiễn:

- **Về lý thuyết:** Cung cấp bằng chứng định lượng về giới hạn chịu tải của hệ ràng buộc cứng, phân biệt rõ tác động của Hard-Busy (ràng buộc tuyệt đối) so với Soft-Busy (ràng buộc mềm có slack).
- **Về thực tiễn:** Giúp bộ phận lập lịch biết được “ngưỡng an toàn” – tức là tỷ lệ cán bộ vắng mặt tuyệt đối tối đa mà hệ thống vẫn có thể lên lịch được.
- **Về tính khả thi:** Đánh giá khả năng chống chịu của mô hình trước các tình huống bất thường (nhiều cán bộ đồng thời xin nghỉ bắt buộc vào kỳ thi).

Thiết kế thí nghiệm

Biến độc lập: Tham số $hard_busy_rate \in \{5\%, 40\%, 80\%, 100\%\}$ – tỷ lệ các ô (i, j) trong ma trận B_{ij} được gán giá trị 2 (Hard-Busy: cấm phân công tuyệt đối). Tham số $soft_busy_rate$ được cố định ở mức 5% xuyên suốt.

Biến phụ thuộc: Trạng thái lời giải của CBC Solver – Optimal (tìm được lịch khả thi tối ưu) hoặc Infeasible (không gian nghiệm rỗng).

Tham số cố định: Toàn bộ dataset gốc ($|CB| = 73$, $|CT| = 83$, $|R| = 3$), trọng số $\omega = 1$, $\theta = 20$, và các hệ số phạt $TAX_LACK_STAFF = 10.000$, $TAX_FORCE_BUSY = 5.000$, $TAX_BAD_QUAL = 5.000$ được giữ nguyên xuyên suốt.

Cơ chế sinh dữ liệu: Hàm `preprocess_data(file_path, soft_busy_rate, hard_busy_rate)` trong `data_preprocessing.py` sử dụng `random.sample` với `seed = 42 + trial` để ngẫu nhiên hóa các ô bận theo đúng tỷ lệ chỉ định. Cache JSON được xóa trước mỗi trial để đảm bảo mỗi mức `hard_busy_rate` được sinh độc lập, không bị ảnh hưởng bởi kết quả trước.

Timeout: Mỗi trial được giới hạn tối đa `SOLVER_TIMEOUT = 5` giây.

Số lần lặp: `NUM_TRIALS = 1` lần mỗi mức – phù hợp với mục tiêu thăm dò ngưỡng định tính.

Công cụ: Script `busy_rate_experiment.py` tự động hóa toàn bộ vòng lặp thí nghiệm, in đồng hồ thời gian thực (LiveTimer) trong khi CBC đang chạy, tính ETA còn lại, và xuất kết quả ra terminal.

Kết quả thí nghiệm

Hard Busy Rate	Optimal	Infeasible	Tỷ lệ Feasible	Thời gian TB (s)
5%	1	0	100,0%	5,18
40%	1	0	100,0%	4,67
80%	0	1	0,0%	1,90
100%	0	1	0,0%	1,93

Bảng 4.27 Kết quả thí nghiệm đo tỷ lệ Feasible/Infeasible theo `hard_busy_rate`

4.4.1 Phân tích và nhận xét

Ngưỡng tối hạn xác định tại 80%: Mô hình duy trì trạng thái **Optimal** ở mức Hard-Busy 5% và 40%, nhưng chuyển sang **Infeasible** khi Hard-Busy đạt 80% và duy trì trạng thái này ở 100%. Điều này xác nhận ngưỡng tối hạn nằm trong khoảng (40%, 80%).

Cơ chế giải thích nằm ở sự khác biệt căn bản giữa hai loại ràng buộc bận trong mô hình. Khi $B_{ij} = 1$ (Soft-Busy), ràng buộc $x_{ijr} \leq s_{ijr}^{\text{busy}}$ vẫn cho phép solver phân công kèm chi phí phạt – mô hình *không bao giờ Infeasible* chỉ vì Soft-Busy tăng. Ngược lại, khi $B_{ij} = 2$ (Hard-Busy), ràng buộc cứng $x_{ijr} = 0$ loại bỏ hoàn toàn cán bộ khỏi ca đó. Khi tỷ lệ Hard-Busy đạt 80%, phần lớn cặp (i, j) bị khóa cứng khiến ràng buộc LeadProctor – yêu cầu ít nhất một người đủ năng lực và không bận tuyệt đối cho mỗi vai trò – không còn có thể thỏa mãn, dẫn đến không gian nghiệm rỗng.

Thời gian giải giảm khi Infeasible: Đáng chú ý, thời gian solver giảm mạnh từ ~5 giây (Optimal) xuống còn ~1,9 giây (Infeasible). Đây là hành vi điển hình của CBC: khi mô hình vô nghiệm, solver phát hiện mâu thuẫn sớm trong quá trình cắt nhánh và kết thúc ngay mà không cần duyệt hết cây tìm kiếm.

Hàm ý thiết kế: Kết quả này làm nổi bật tầm quan trọng của việc phân tầng ràng buộc bận thành Hard và Soft. Với thiết kế hai lớp hiện tại, hệ thống vừa đảm bảo tính linh hoạt vận hành (Soft-Busy luôn cho phép tìm nghiệm) vừa tôn trọng các trường hợp vắng mặt bắt buộc thực sự (Hard-Busy). Ngưỡng an toàn $\leq 40\%$ Hard-Busy cung cấp một tham chiếu định lượng cụ thể cho bộ phận quản lý nhân sự trong quá trình lập kế hoạch kỳ thi.

Kết luận: Ngưỡng tối hạn của mô hình nằm trong khoảng Hard-Busy $\in (40\%, 80\%)$. Trong điều kiện vận hành thực tế, tỷ lệ cán bộ vắng mặt tuyệt đối hiếm khi vượt 20–30%, do đó mô hình hoàn toàn đáp ứng được yêu cầu robustness của bài toán phân công cán bộ coi thi.

4.5 Khảo sát tập tối ưu Pareto và xác định hệ số phạt

4.5.1 Lý do khảo sát

Hàm mục tiêu của mô hình ILP được xây dựng theo dạng tổng hợp tuyến tính:

$$\min \quad \omega \cdot F_2 + \theta \cdot F_1$$

trong đó $F_1 = W_{\max} - W_{\min}$ đo độ lệch công bằng về số ca thi (nhỏ hơn là tốt hơn) và F_2 là tổng điểm phạt chất lượng lịch (nhỏ hơn là tốt hơn). Hai thành phần này mâu thuẫn nhau về bản chất: ép công bằng tuyệt đối ($\theta \rightarrow \infty$) buộc solver phân phối đều ca thi cho mọi cán bộ, nhưng lại bỏ qua các yếu tố chất lượng như sở thích cơ sở và mệt mỏi; ngược lại, ưu tiên chất lượng ($\theta \rightarrow 0$) dẫn đến lịch bất công khi một số cán bộ gánh quá nhiều ca.

Vì không tồn tại một cặp (ω, θ) nào là *tối ưu tuyệt đối* cho mọi hoàn cảnh, việc khảo sát hệ thống các giá trị θ là cần thiết để:

- Định lượng đường cong đánh đổi (trade-off curve) giữa F_1 và F_2 .
- Cung cấp bằng chứng định lượng để bộ phận lập lịch lựa chọn mức ưu tiên công bằng phù hợp với chính sách của trường.

4.5.2 Tập tối ưu Pareto là gì?

Trong bài toán tối ưu đa mục tiêu, một nghiệm A được gọi là **thống trị** (dominate) nghiệm B nếu A tốt hơn hoặc bằng B trên *mọi* tiêu chí, và tốt hơn B trên *ít nhất một* tiêu chí. **Tập tối ưu Pareto** (Pareto Frontier) là tập hợp tất cả các nghiệm không bị thống trị bởi bất kỳ nghiệm nào khác – tức là mọi sự cải thiện thêm về một tiêu chí đều buộc phải hi sinh tiêu chí còn lại.

Trong ngữ cảnh bài toán này, mỗi điểm trên Pareto Frontier tương ứng với một giá trị θ cụ thể, xác định một lịch phân công với cặp $(F_1, \text{Violations})$ mà không có lịch nào khác đồng thời công bằng hơn và ít vi phạm hơn. Điểm **Knee Point** là điểm cân bằng tốt nhất và được đề xuất làm mặc định.

4.5.3 Phương pháp khảo sát

Thiết kế thí nghiệm

Nhóm xây dựng script `pareto_search.py` để tự động hóa toàn bộ quy trình Grid Search. Cố định $\omega = 1,0$ và biến thiên θ qua lưới giá trị:

$$\theta \in \{20, 40, 60, 80, 100, 200, 500, 1000, 10000\}$$

Lý do chỉ biến thiên θ : thành phần F_2 có giá trị thô lên tới hàng nghìn trong khi F_1 chỉ dao động từ 0 đến ~20 ca. Cố định $\omega = 1,0$ và tăng θ là cách chuẩn hóa tương quan lực giữa hai thành phần mà không cần thay đổi đơn vị đo. Các tham số còn lại giữ nguyên xuyên suốt: $\text{TAX_LACK_STAFF} = 10.000$, $\text{TAX_FORCE_BUSY} = 5.000$, $\text{TAX_BAD_QUAL} = 5.000$, $\text{timeLimit} = 60$ giây, $\text{gapRel} = 2\%$.

Chỉ số đánh giá mỗi điểm

Với mỗi giá trị θ , nhóm thu thập hai chỉ số cốt lõi:

- **Gap ca thi** ($F_1 = W_{\max} - W_{\min}$): độ lệch số ca giữa cán bộ bận nhất và nhàn nhất.
- **Tổng vi phạm**: tổng hợp có trọng số = $\text{slack_cap} \times 3 + \text{slack_busy} \times 1 + \text{slack_qual} \times 2 + \text{fatigue_count} \times 1 + \text{travel_count} \times 1$

Thuật toán xác định Knee Point

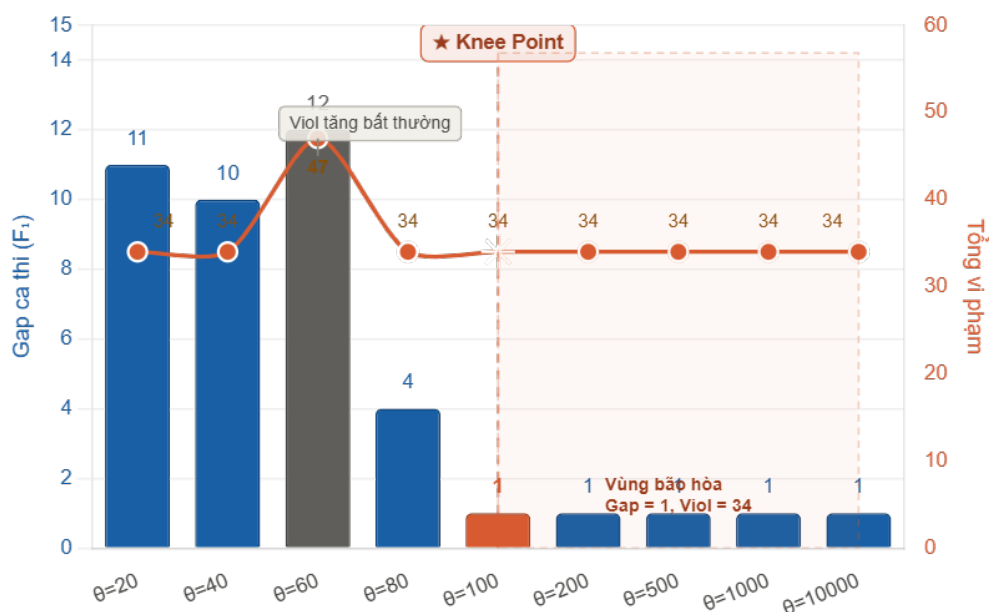
Sau khi thu thập toàn bộ kết quả, nhóm áp dụng quy trình:

1. Lọc các điểm Pareto-optimal (không bị thống trị).
2. Chuẩn hóa F_1 và Violations về $[0, 1]$.
3. Tính điểm cân bằng: $\text{balance_score} = \sqrt{F_{1,\text{norm}}^2 + \text{Viol}_{\text{norm}}^2}$.
4. Điểm có balance_score nhỏ nhất là Knee Point – điểm được đề xuất.

4.5.4 Kết quả khảo sát

θ	Trạng thái	Gap (ca)	Tổng vi phạm	Ghi chú
20	Feasible	11,0	34	Mặc định ban đầu
40	Feasible	10,0	34	
60	Feasible	12,0	47	Gap và Viol đều tăng
80	Optimal	4,0	34	Bước nhảy đột biến về Gap
100	Feasible	1,0	34	★ Knee Point – Đề xuất
200	Optimal	1,0	34	Tương đương $\theta = 100$
500	Feasible	1,0	34	Vùng bão hòa
1000	Feasible	1,0	34	Vùng bão hòa
10000	Feasible	1,0	34	Vùng bão hòa

Bảng 4.29 Kết quả Grid Search Pareto theo θ (cố định $\omega = 1,0$)



Hình 4.6 Biểu đồ Pareto

4.5.5 Phân tích và nhận xét

Giai đoạn $\theta \in [20, 60]$: Gap dao động không đơn điệu ($11 \rightarrow 10 \rightarrow 12$ ca), trong khi tổng vi phạm tăng lên 47 ở $\theta = 60$. Đây là vùng tham số kém hiệu quả – θ chưa đủ lớn để định hướng rõ ràng cho solver.

Bước nhảy tại $\theta = 80$: Gap giảm đột biến từ 12 xuống 4 ca trong khi vi phạm giữ nguyên ở 34. Đây là ngưỡng mà trọng số $\theta \cdot F_1$ đủ mạnh để solver ưu tiên công bằng.

Giai đoạn $\theta \geq 100$ (vùng bão hòa): Gap ổn định ở mức 1 ca và vi phạm không thay đổi (34). Tăng thêm θ không mang lại cải thiện về cả hai tiêu chí, chỉ làm tăng thời gian giải.

Xác định Knee Point và hệ số đề xuất

Điểm $\theta = 100$ được xác định là **Knee Point**: là điểm đầu tiên đạt vùng bão hòa (Gap = 1 ca, Vi phạm = 34) với giá trị θ nhỏ nhất trong nhóm, giúp lịch đạt được công bằng nhất và giảm tối đa sự mệt mỏi hay thời gian chờ cho cán bộ. Điểm $\theta = 200$ tuy đạt Optimal song cho kết quả hoàn toàn tương đồng, nên không mang lại lợi ích bổ sung.

Tham số	Giá trị được đề xuất
ω	1,0
θ	100,0
TAX_LACK_STAFF	10.000
TAX_FORCE_BUSY	5.000
TAX_BAD_QUAL	5.000

Bảng 4.30 Bộ tham số được đề xuất sau khảo sát Pareto

Kết luận: Kết quả khảo sát xác nhận bộ tham số mặc định ban đầu ($\theta = 20$) là *kém hiệu quả* – Gap lên tới 11 ca trong khi chỉ cần tăng θ lên 100 để đưa Gap xuống 1 ca mà không làm tăng vi phạm. Ngưỡng $\theta = 100$ được chọn làm giá trị mặc định cho toàn bộ các thử nghiệm so sánh tiếp theo. Đối với các trường có chính sách ưu tiên công bằng tuyệt đối, bất kỳ giá trị $\theta \in [100, 1000]$ đều cho kết quả tương đương và được chấp nhận.

5 Đề xuất mô hình mở rộng trong tương lai (Future Extension)

Mặc dù mô hình Quy hoạch Nguyên Tuyến tính (ILP) kết hợp với bộ giải CBC đã giải quyết xuất sắc bài toán phân công cán bộ coi thi (IAP) trên tập dữ liệu hiện tại, phân tích độ phức tạp ở Chương 4 đã chỉ ra rằng IAP là một bài toán thuộc lớp NP-Hard. Thời gian giải quyết của hệ thống sẽ tăng theo cấp số nhân $O(2^V)$ khi quy mô bài toán được mở rộng.

Trong thực tế tại ĐH Bách Khoa TP.HCM, nếu áp dụng hệ thống này cho toàn bộ kỳ thi của tất cả các khoa (quy mô lên tới hàng nghìn cán bộ và hàng vạn ca thi), bộ giải ILP truyền thống sẽ rơi vào tình trạng bùng nổ tổ hợp (Combinatorial Explosion) và không thể tìm ra nghiệm trong thời gian cho phép. Do đó, nhóm đề xuất các hướng tiếp cận thay thế (Alternative Approaches) nhằm giải quyết bài toán ở quy mô lớn (Large-scale scaling) như sau:

5.1 Giải pháp sử dụng Thuật toán Di truyền (Genetic Algorithm - GA)

Thuật toán Di truyền (GA) là một phương pháp tối ưu hóa Meta-heuristic lấy cảm hứng từ quá trình tiến hóa tự nhiên. Thay vì tìm kiếm nghiệm tối ưu tuyệt đối (Optimal) như ILP, GA hướng tới việc tìm ra nghiệm "đủ tốt" (Near-optimal) trong một khoảng thời gian ngắn và cố định.

- Biểu diễn nhiễm sắc thể (Chromosome): Mỗi phương án phân công lịch thi của

toàn trường sẽ được mã hóa thành một chuỗi nhiễm sắc thể. Trong đó, mỗi gen đại diện cho một ca thi, chứa thông tin về các cán bộ được gán vào ca đó.

- Hàm thích nghi (Fitness Function): Hàm mục tiêu tổng hợp Z (bao gồm công bằng khối lượng F_1 và tổng điểm phạt F_2) sẽ được nghịch đảo để làm hàm đánh giá độ thích nghi. Lịch thi nào vi phạm ít ràng buộc mềm hơn sẽ có điểm Fitness cao hơn.
- Toán tử lai ghép và đột biến (Crossover & Mutation): Các lịch phân công tốt sẽ được "lai ghép" với nhau để tạo ra thế hệ lịch mới ưu việt hơn. Toán tử "đột biến" có thể được thiết kế để hoán đổi ngẫu nhiên ca gác thi giữa 2 cán bộ, giúp thuật toán thoát khỏi các điểm tối ưu cục bộ (Local Optima).
- Ưu điểm: Khả năng tính toán song song rất mạnh, có thể mở rộng xử lý cho toàn trường mà không lo tràn bộ nhớ hay treo hệ thống.

Bên cạnh đó, do Python có thể gặp giới hạn về tốc độ xử lý khi quần thể tiến hóa (population) phình to, hệ thống giải thuật GA trong tương lai có thể được triển khai bằng ngôn ngữ lập trình hiệu năng cao (như C++) kết hợp với việc xây dựng các cấu trúc dữ liệu tùy chỉnh tối ưu. Điều này sẽ giúp hệ thống quản lý, tìm kiếm và truy xuất lịch thi đột biến một cách tức thời, triệt tiêu tối đa độ trễ bộ nhớ.

5.2 Giải pháp Tìm kiếm Lân cận (Local Search) và Luyện kim nhân tạo (Simulated Annealing - SA)

Đối với mô hình phân công có cấu trúc chặt chẽ như IAP, thuật toán Luyện kim nhân tạo (SA) là một hướng đi cực kỳ hứa hẹn.

- Hệ thống sẽ bắt đầu từ một lịch phân công gốc (Baseline) hoặc một lịch được tạo ngẫu nhiên nhưng thỏa mãn các ràng buộc cứng.
- Sau đó, hệ thống thực hiện các bước di chuyển lân cận (Neighborhood Moves) như: gỡ bỏ một cán bộ khỏi ca này và đắp sang ca khác, hoặc trao đổi ca thi giữa 2 người.
- Nhiệt độ (Temperature) trong thuật toán SA cho phép hệ thống đôi khi chấp nhận một lịch tồi hơn ở những bước đầu để tránh bị kẹt ở điểm tối ưu cục bộ, sau đó "nguội dần" để hội tụ về phương án phân công tối ưu nhất.

5.3 Phương pháp kết hợp Matheuristic (ILP + Heuristic)

Thay vì từ bỏ hoàn toàn sức mạnh tính toán chính xác của ILP, nhóm đề xuất một mô hình lai (Hybrid).

- Hệ thống có thể chia nhỏ tập dữ liệu khổng lồ thành các cụm (Clustering) nhỏ hơn (ví dụ: giải bài toán phân công nội bộ riêng cho từng Khoa trước).
- Sử dụng solver ILP để giải nhanh và tối ưu tuyệt đối cho các bài toán con này.
- Sau đó, sử dụng các thuật toán Heuristic để "vá" và điều phối lại những cán bộ dư thừa ở các Khoa với nhau. Cách này vừa giữ được tính tối ưu toán học, vừa giải quyết được bài toán thời gian khi mở rộng quy mô.

5.4 Tối ưu hóa Hàm phạt thời gian chờ (Waiting Penalty Function)

Trong phiên bản mô hình ILP hiện tại, hàm tính điểm phạt thời gian chờ liên ca P^{wait} đang được thiết kế tăng tuyến tính đối với mọi khoảng thời gian trống từ 2 giờ trở lên ($gap \geq 2$). Tuy nhiên, về mặt hành vi tâm lý thực tế, mức độ bất tiện của cán bộ coi thi không tăng tuyến tính mãi theo thời gian.

Cụ thể, việc kẹt lại trường với khoảng trống 3-4 tiếng thường gây ra mức độ ức chế cao nhất (do lỡ cỡ thời gian). Ngược lại, nếu khoảng trống kéo dài từ 5-6 tiếng trở lên, cán bộ có xu hướng quyết định di chuyển về nhà để nghỉ ngơi hoặc giải quyết việc cá nhân, từ đó làm giảm đáng kể sự mệt mỏi do phải chờ đợi thụ động tại cơ sở.

Hướng khắc phục: Trong các phiên bản mở rộng của hệ thống, nhóm đề xuất thay thế hàm P^{wait} tuyến tính bằng một trong hai mô hình toán học sau nhằm phản ánh chính xác hơn hành vi ra quyết định của con người:

- **Hàm phạt hình chuông (Bell-curve/Gaussian function):** Điểm phạt sẽ tăng dần và đạt đỉnh cực đại (Peak) ở mốc chờ khoảng 3-4 giờ, sau đó tiệm cận giảm dần khi thời gian chờ tiếp tục kéo dài qua mốc 6 tiếng.
- **Hàm bậc thang giảm dần (Step-down function):** Thiết lập các ngưỡng (threshold) ngắt quãng. Khi khoảng trống vượt qua ngưỡng 5 giờ, hệ số phạt sẽ tự động rút xuống một mức thấp hơn và duy trì ổn định, thay vì tiếp tục cộng dồn tiến tới vô cực như mô hình đường thẳng tuyến tính.

Sự tương thích kiến trúc: Về bản chất toán học tối ưu, việc nhúng các hàm phi tuyến (hình chuông, bậc thang) vào hệ thống ILP truyền thống sẽ đòi hỏi kỹ thuật tuyến tính hóa từng khúc (Piecewise Linearization), làm gia tăng đột biến số lượng biến nhị phân và làm trầm trọng thêm vấn đề bùng nổ tổ hợp. Tuy nhiên, nếu hệ thống được nâng cấp sang kiến trúc Meta-heuristic (GA hoặc SA) như đã đề xuất ở Mục 5.1 và 5.2, việc cài đặt các hàm phạt phi tuyến này vào Hàm thích nghi (Fitness Function) lại cực kỳ đơn giản và đạt hiệu năng truy xuất $O(1)$. Điều này cho thấy sự đồng bộ và tất yếu của việc chuyển đổi mô hình khi giải quyết bài toán ở quy mô lớn.

6 Tổng kết và Bài học kinh nghiệm

Qua quá trình nghiên cứu, thiết kế và thực nghiệm mô hình hóa cho bài toán phân công giám thị (Invigilator Assignment Problem - IAP), tập thể nhóm đã đúc kết được những bài học mang tính nền tảng, không chỉ về mặt kỹ thuật thuật toán mà còn về tư duy hệ thống khi chuyển đổi một bài toán điều độ thực tế thành một mô hình tối ưu hóa toán học tường minh.

6.1 Sự trưởng thành trong tư duy phân hoạch ràng buộc

Một trong những bước ngoặt quan trọng nhất trong quá trình phát triển mô hình là việc nhận thức và tái định hình ranh giới giữa các ràng buộc cứng (hard constraints) và ràng buộc mềm (soft constraints). Trong thiết kế ban đầu, do mong muốn tối đa hóa không gian nghiệm khả thi và tăng tính linh hoạt cho thuật toán, nhóm đã thực hiện nới lỏng diện rộng đối với các nhóm ràng buộc cốt lõi như trạng thái bận của cán bộ, yêu cầu năng lực chuyên môn và tính đầy đủ của phân công. Việc lạm dụng cơ chế nới lỏng này đã dẫn đến hiện tượng sai lệch nghiêm trọng về mặt logic toán học: khi kiểm thử tại các kịch bản cực đoan (biên nghiệm đặc biệt) — ví dụ như trường hợp giả định 100% cán bộ đều bận — mô hình vẫn tìm ra và trả về một phương án phân công tối ưu cục bộ thay vì phải trả về trạng thái vô nghiệm (Infeasible).

Để khắc phục lỗ hổng logic này, mô hình sau cùng đã được chuẩn hóa lại theo một cấu trúc phân cấp ràng buộc chặt chẽ và khoa học:

- **Hệ ràng buộc cứng tuyệt đối (Strict Hard Constraints):** Định hình ranh giới không thể xâm phạm của không gian vật lý, bảo đảm tính đúng đắn tối thiểu của lịch trình (ví dụ: một cán bộ không thể xuất hiện tại hai địa điểm trong cùng một khung giờ giao thoa). Việc vi phạm các ràng buộc này đồng nghĩa với việc bài toán toán học không tồn tại nghiệm khả thi.
- **Hệ ràng buộc mềm chế tài cao (Heavy-Penalized Soft Constraints):** Áp dụng cho các chỉ tiêu như tính khả dụng mềm ($B_{ij} = 1$) hoặc độ lệch chuyên môn. Việc vi phạm sẽ không làm sụp đổ mô hình mà cấu thành các điểm phạt nặng (Heavy Penalty) tích hợp trực tiếp vào hàm mục tiêu, định hướng bộ giải (Solver) tìm kiếm các vùng không gian nghiệm tối ưu hơn mà vẫn phản ánh đúng bản chất khắc nghiệt của dữ liệu đầu vào.

6.2 Bài học về kiến trúc hệ thống và kiểm toán dữ liệu tĩnh

Bên cạnh khía cạnh toán học thuần túy, quá trình triển khai mã nguồn trên ngôn ngữ Python kết hợp thư viện pulp đã mang lại cho nhóm góc nhìn thực tiễn về kỹ thuật phần mềm trong tối ưu hóa. Việc phát triển phân hệ kiểm toán tiền khả thi (*Static Feasibility Audit*) trước khi kích hoạt bộ giải là một minh chứng cụ thể. Việc phát hiện sớm các điểm nghẽn về mặt cung - cầu nhân sự tại từng khung giờ giao thoa không chỉ giúp ngăn chặn việc lãng phí tài nguyên tính toán của Solver đối với các bài toán vô nghiệm hiển nhiên, mà còn tạo ra một cơ chế tương tác trực quan, cho phép người vận hành điều chỉnh dữ liệu một cách linh hoạt (Manual Data Adjustment).

6.3 Kết luận về mặt năng lực thu nhận

Tóm lại, bài tập lớn này đã giúp nhóm tiếp thu và làm chủ một quy trình toàn diện từ khía cạnh lý thuyết đến thực nghiệm cấu trúc: từ tiền xử lý dữ liệu thô, phân tích tham số, thiết lập hệ phương trình tuyến tính nguyên (ILP) đến kỹ thuật tinh chỉnh trọng số (Weight Tuning) dựa trên lịch sử hội tụ. Sự chuyển biến từ một mô hình ban đầu còn lỏng lẻo sang một hệ thống phân bổ có cấu trúc đa tầng chặt chẽ đã chứng minh tầm quan trọng của việc tư duy biện chứng giữa toán học lý thuyết và tính khả thi trong bài toán quản trị vận hành thực tế.



A Biên bản làm việc nhóm

ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA
VIỆT NAM

Độc lập – Tự do – Hạnh phúc

BIÊN BẢN LÀM VIỆC NHÓM
MÔN: MÔ HÌNH HOÁ TOÁN HỌC
LỚP: L01 — Nhóm: 9

Buổi họp số: 01

Ngày họp: 10/05/2026

Thời gian: 20:00

Địa điểm/Hình thức: Online qua Google Meet

Chủ trì (nhóm trưởng): Nguyễn Duy Khánh

Thư ký (người ghi biên bản): Trần Quỳnh Anh

Nội dung:

1. PHÂN CÔNG NHIỆM VỤ:

STT	HỌ VÀ TÊN	NHIỆM VỤ	THỜI HẠN
1	Nguyễn Duy Khánh	Định hướng mô hình: - Phần 1: Phân tích tham số. - Phần 2: Biến mục tiêu. - Phần 3: Hàm mục tiêu. - Viết báo cáo 1, 2, 3, kết luận. - Kiểm tra tổng thể.	18/05/2026
2	Phan Quốc Huy	Mô hình hóa, làm slide: - Phần 4: Hàm ràng buộc cứng. - Phần 5: Hàm ràng buộc mềm. - Phần 10: Thuật toán đề xuất. - Làm slide, hỗ trợ làm phần sau latex.	- Mô hình hóa: 18/05/2026 - Phần 10, Làm slide: 22/05/2026

STT	HỌ VÀ TÊN	NHIỆM VỤ	THỜI HẠN
3	Nguyễn Ngọc Anh Khoa	Code Python chủ đạo: - Phần 6: Đọc dataset, xử lý data. - Phần 7: Xây dựng mô hình ILP tại các tool được đề xuất, tích hợp solver vào code. - Phần 8: Chạy solver, tuning nếu infeasible. - Viết tuning: xử lý khi solver báo lỗi. - Viết README.md, requirements.txt.	18/05/2026
4	Trần Quỳnh Anh	Phân tích hiệu năng, viết Meeting minutes: - Phần 9: Đánh giá độ phức tạp, thời gian thực thi, so sánh với baseline. - Viết phần Appendix, Meeting minutes.	20/05/2026

2. QUÁ TRÌNH THỰC HIỆN VÀ THẢO LUẬN

- **Khánh:** Phân tích đề bài và các yêu cầu cần thực hiện.
- **Khánh:** Đưa ra các lựa chọn về nhiệm vụ, hỏi thành viên có hợp lý hay không.
- **Thành viên:** Chấp thuận cách chia nhiệm vụ.
- **Khánh:** Đề xuất các thành viên chọn nhiệm vụ.
- **Thành viên:** Chọn nhiệm vụ.
- **Khánh:** Hỏi mọi người còn có ý kiến gì không.
- **Khoa:** Thắc mắc về hàm mục tiêu và ràng buộc.
- **Khánh:** Phân tích sơ bộ hàm mục tiêu và ràng buộc mà nhóm sẽ chọn.

3. KẾT LUẬN

Nhóm đã thống nhất được hàm mục tiêu, ràng buộc và nhiệm vụ cụ thể cho từng thành viên.

Cuộc họp kết thúc vào 21 giờ 00, ngày 10 tháng 05 năm 2026 .

THƯ KÝ

Trần Quỳnh Anh

NHÓM TRƯỞNG

Nguyễn Duy Khánh

Tài liệu tham khảo

- [1] F. Hanum, M. A. Romliyah, và T. Bakhtiar, “Exam invigilators assignment problem: A goal programming approach,” *Applied Mathematical Sciences*, vol. 9, no. 58, tr. 2871–2880, 2015.
- [2] M. I. Hanafi, S. S. Ali, R. M. Jusoh, F. Ali, và N. A. Rahman, “A Mixed Integer Linear Programming for Exam-Invigilator Assignment Problem: A Case Study at Universiti Pertahanan Nasional Malaysia,” *JOIV : International Journal on Informatics Visualization*, vol. 8, no. 2, tr. 686–691, 2024.
- [3] M. Çimen, S. Belbağ, M. Soysal, và Ç. Sel, “Invigilators assignment in practical examination timetabling problems,” *International Journal of Industrial Engineering: Theory, Applications and Practice*, vol. 29, no. 3, 2022.